# CpE 3110
# Computer Organization & Design

Dr. Minsu Choi

The Processor

# CHAPTER 4

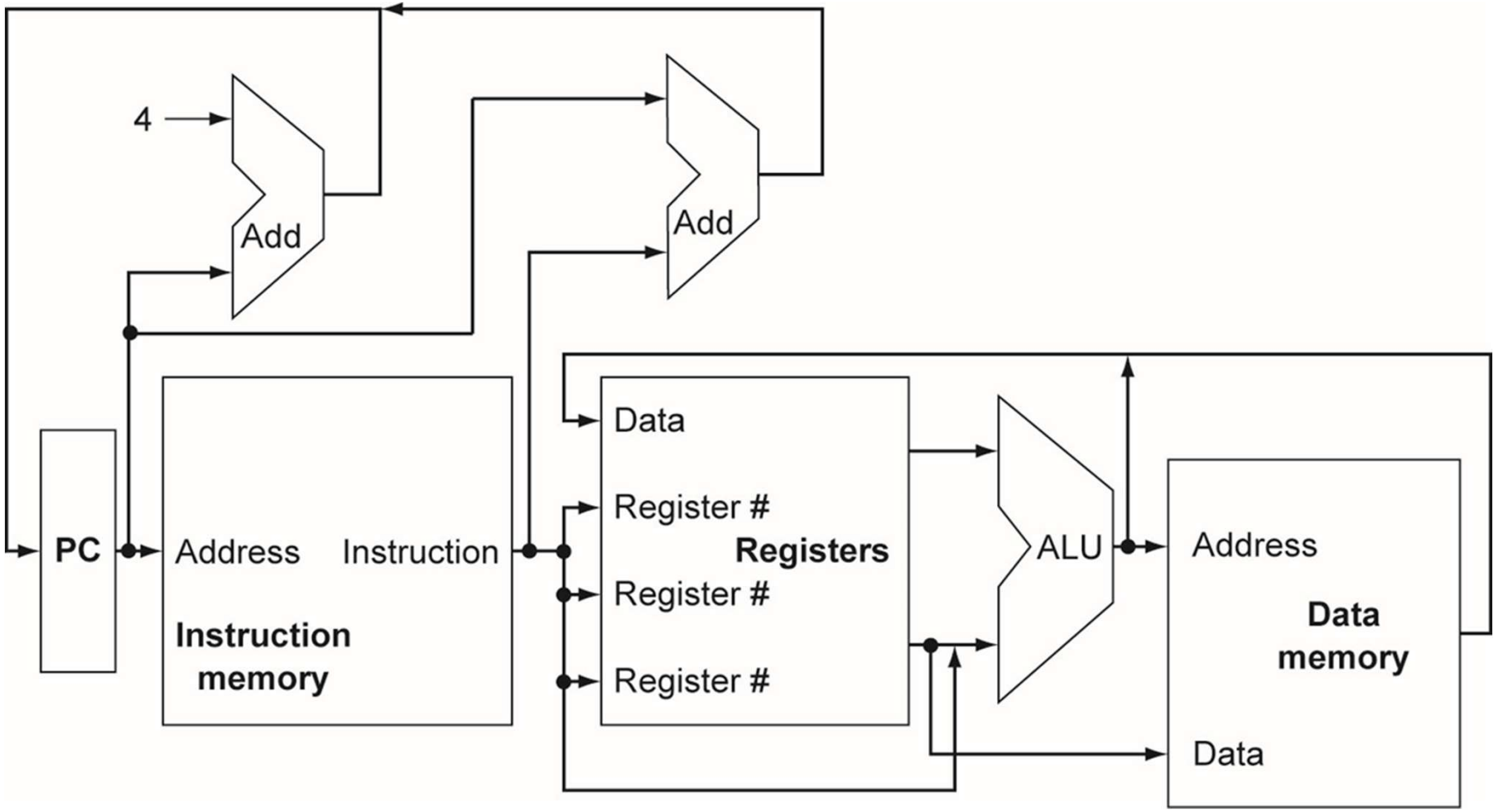
# Introduction

- CPU performance factors
  - Instruction count
    - Determined by ISA and compiler
  - CPI and Cycle time
    - Determined by CPU hardware
- We will examine two LEGv8 implementations
  - A simplified version (single cycle implementation)
  - A more realistic pipelined version
- Simple subset, shows most aspects
  - Memory reference: LDUR, STUR
  - Arithmetic/logical: ADD, SUB, AND and ORR
  - Control transfer: CBZ and B

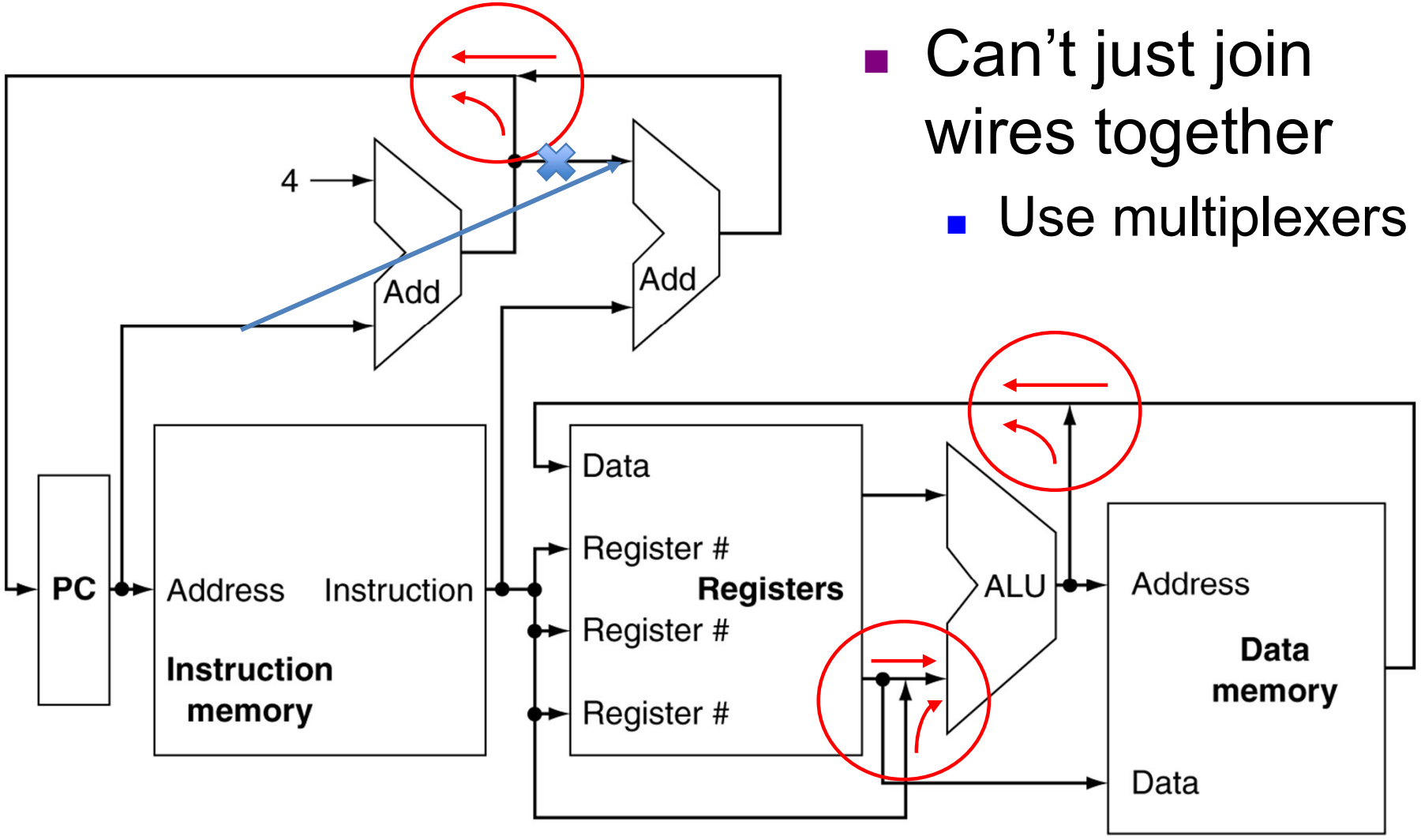
# Instruction Execution Cycle

- Fetch → Decode → Execute → Fetch…
- PC → instruction memory, fetch instruction
- Register numbers → register file, read registers
- Depending on instruction class
  - Use ALU to calculate
    - Arithmetic result
    - Memory address for load/store
    - Branch target address
  - Access data memory for load/store
  - PC ← target address or PC + 4

# CPU Overview

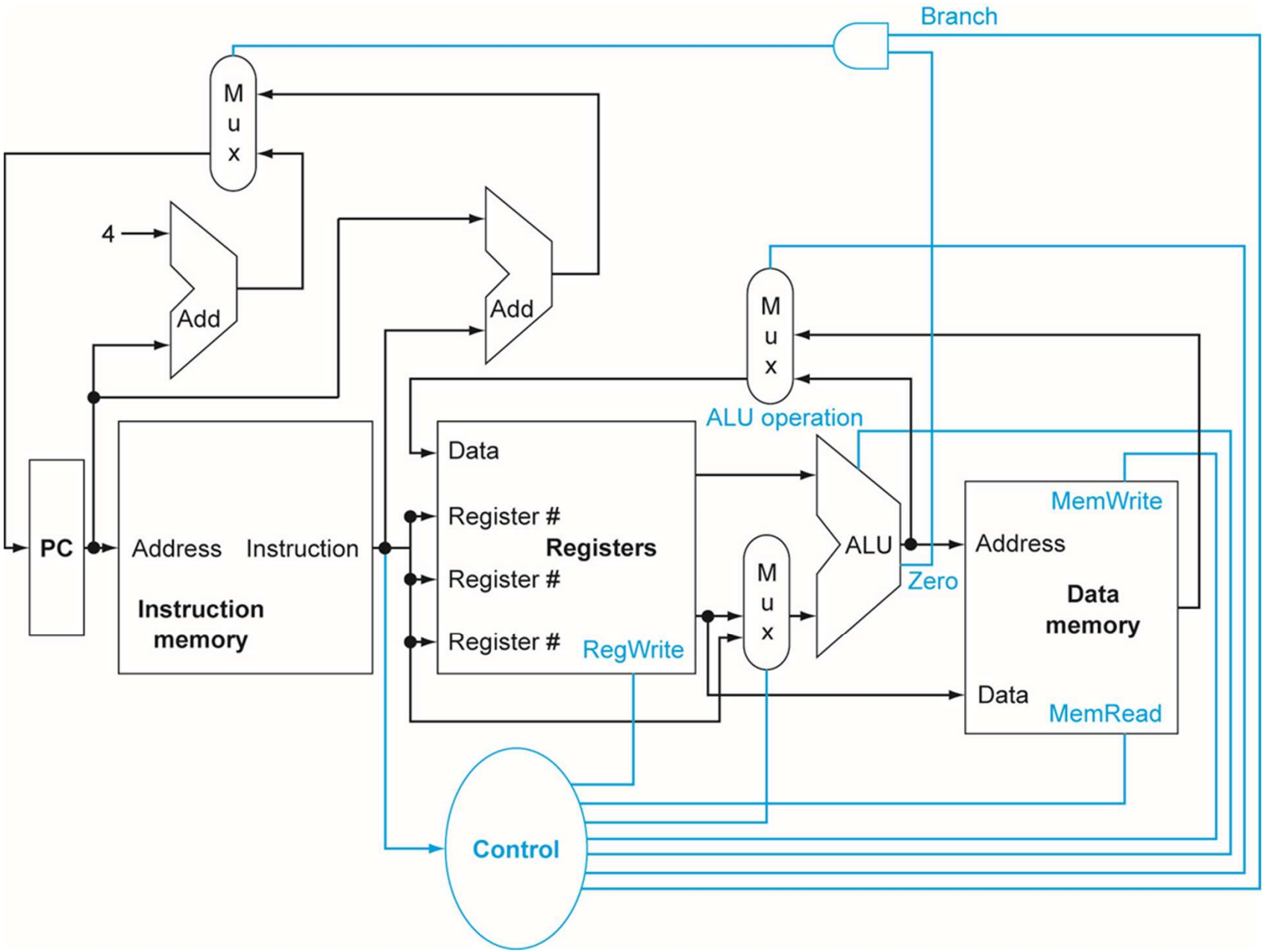


# Multiplexers




- Can't just join wires together
  - Use multiplexers

# Control


Branch
M
u
x
4
Add
Add
M
u
x
ALU operation
PC
Address
Instruction
Instruction
memory
Data
Register #
Registers
Register #
Register #
RegWrite
M
u
x
ALU
Zero
MemWrite
Address
Data
memory
Data
MemRead
Control

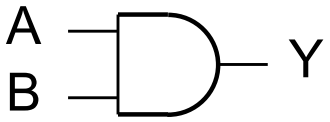
# Logic Design Basics



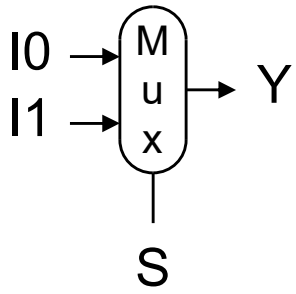
- Information encoded in binary
  - Low voltage = 0, High voltage = 1
  - One wire per bit
  - Multi-bit data encoded on multi-wire buses
- Combinational element
  - Operate on data
  - Output is a function of input
- State (sequential) elements
  - Store information

# Combinational Elements

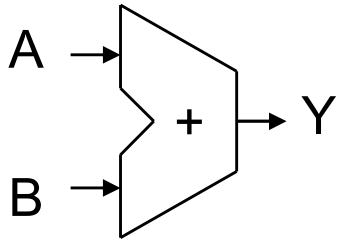
- AND-gate
  - Y = A & B



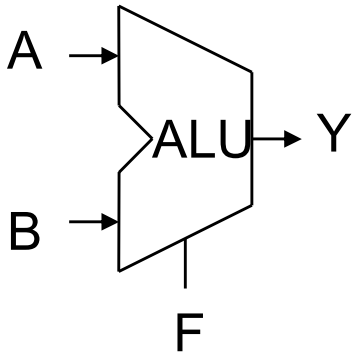

- Multiplexer
  - Y = S ? I1 : I0




- Adder
  - Y = A + B

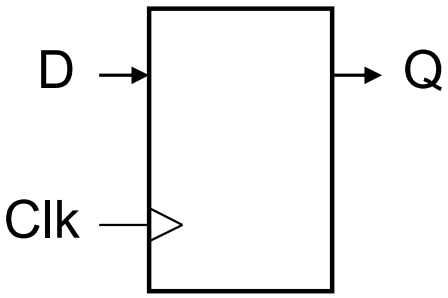


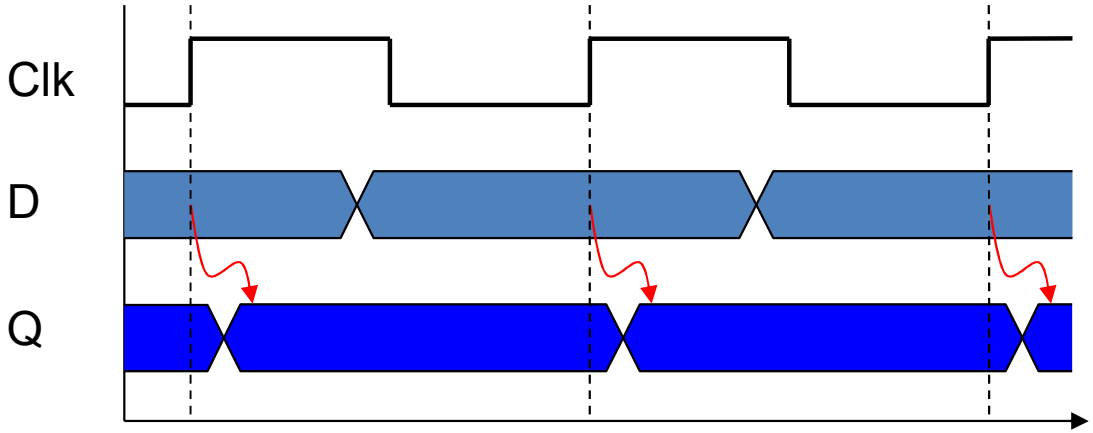

- Arithmetic/Logic Unit
  - Y = F(A, B)



# Sequential Elements

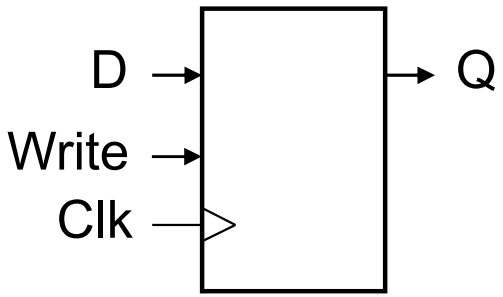
- Register: stores data in a circuit
  - Uses a clock signal to determine when to update the stored value
  - Edge-triggered: update when Clk changes from 0 to 1

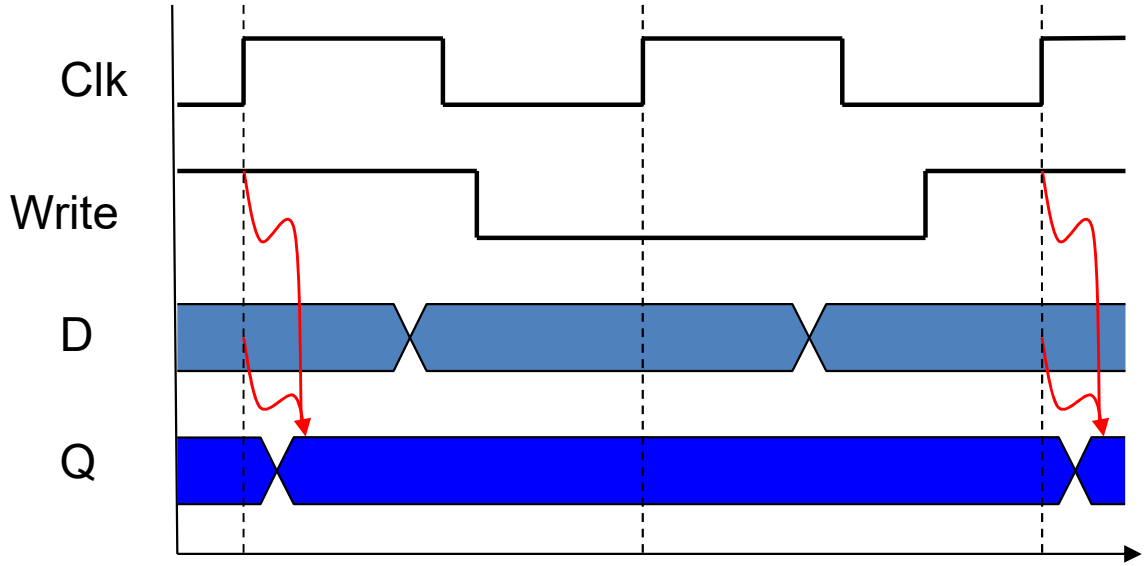


# Sequential Elements

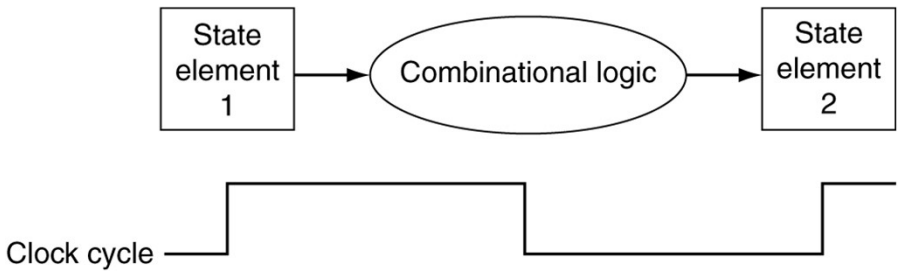
- Register with write control
  - Only updates on clock edge when write control input is 1
  - Used when stored value is required later

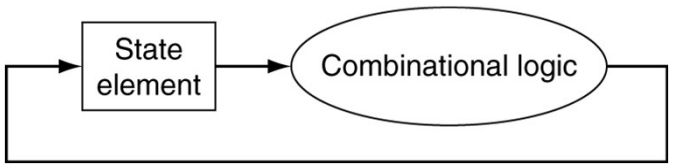


# Clocking Methodology

- Combinational logic transforms data during clock cycles
  - Between clock edges
  - Input from state elements, output to state element
  - Longest delay determines clock period



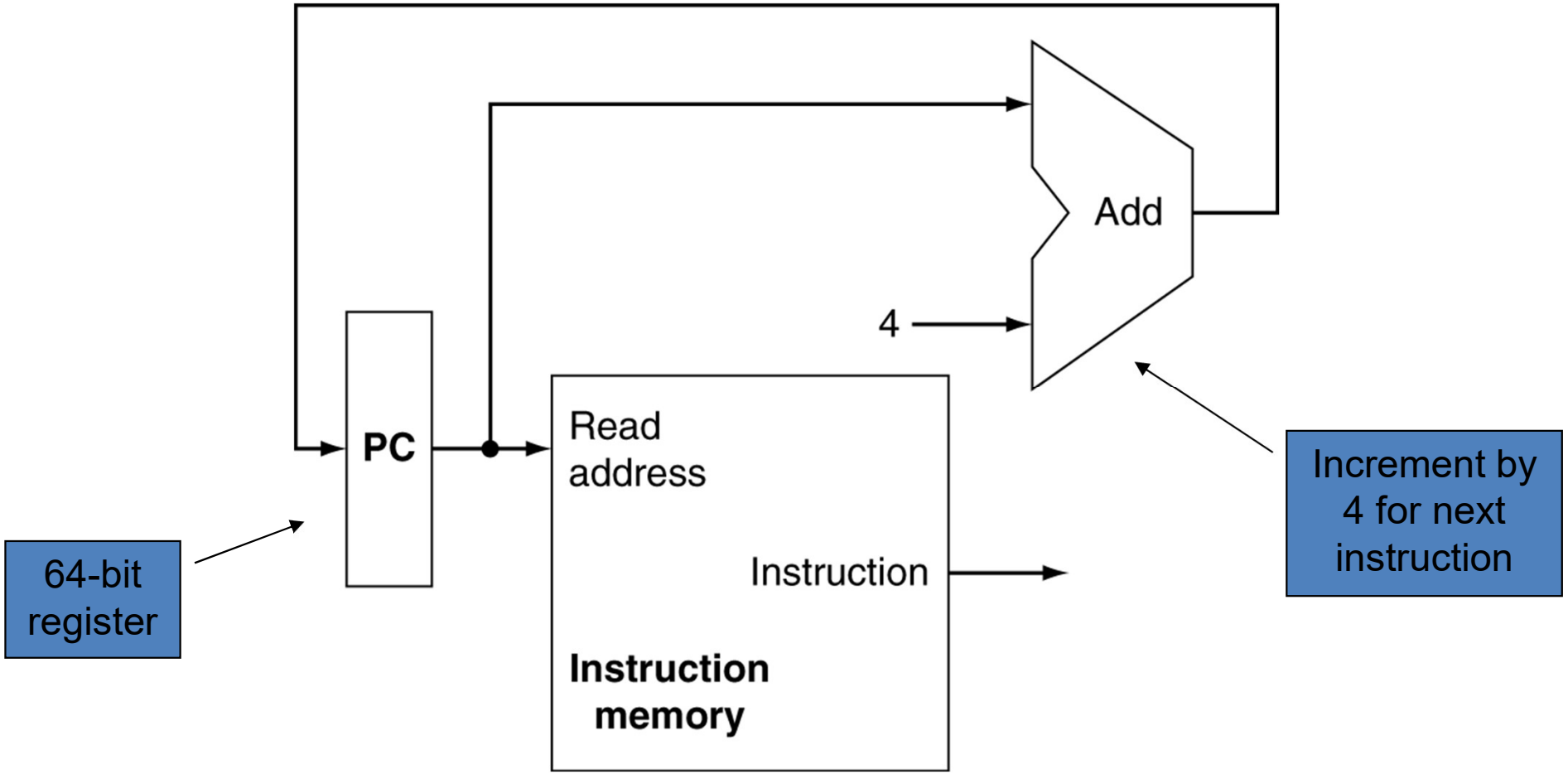



# Building a Datapath



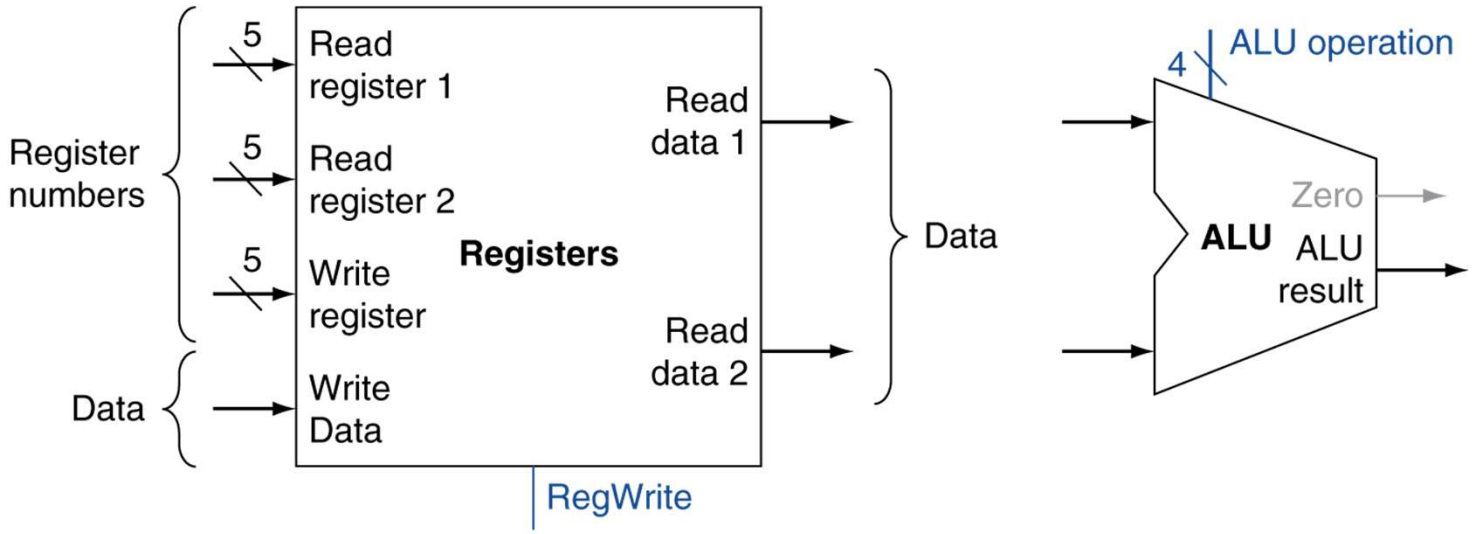
- Datapath
  - Elements that process data and addresses in the CPU
    - Registers, ALUs, mux's, memories, …
- We will build a LEGv8 datapath incrementally
  - Refining the overview design

# Instruction Fetch



# R-Format Instructions

- Read two register operands
- Perform arithmetic/logical operation
- Write register result

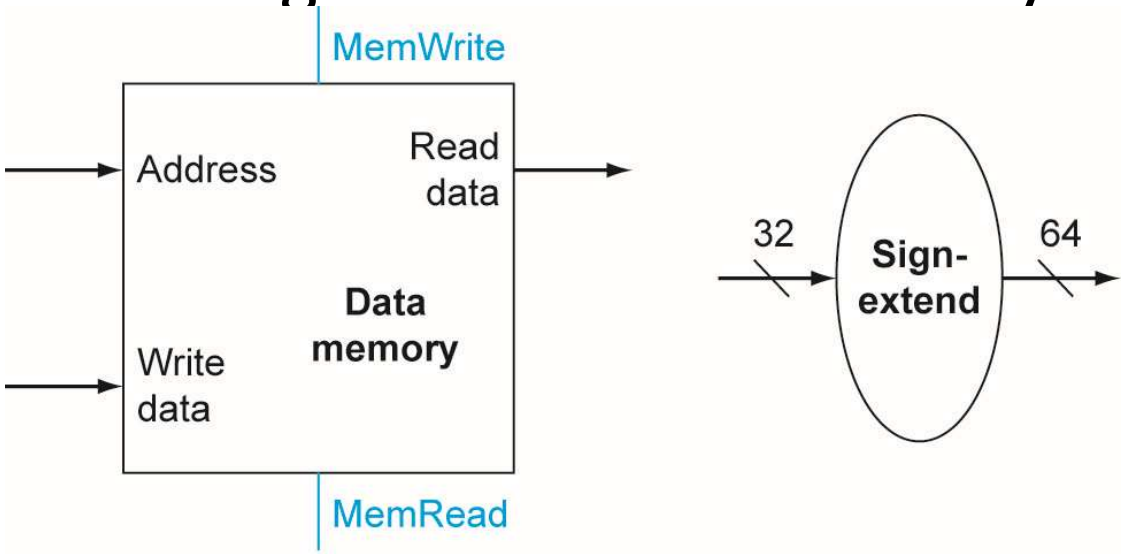



a. Registers

b. ALU

# Load/Store Instructions

- Read register operands
- Calculate address using 9-bit offset
  - Use ALU, but sign-extend offset
  - Sign-extension unit takes 32-bit instruction. Then, selects 9b offset for load/store (19b offset for CBZ).
- Load: Read memory and update register
- Store: Write register value to memory



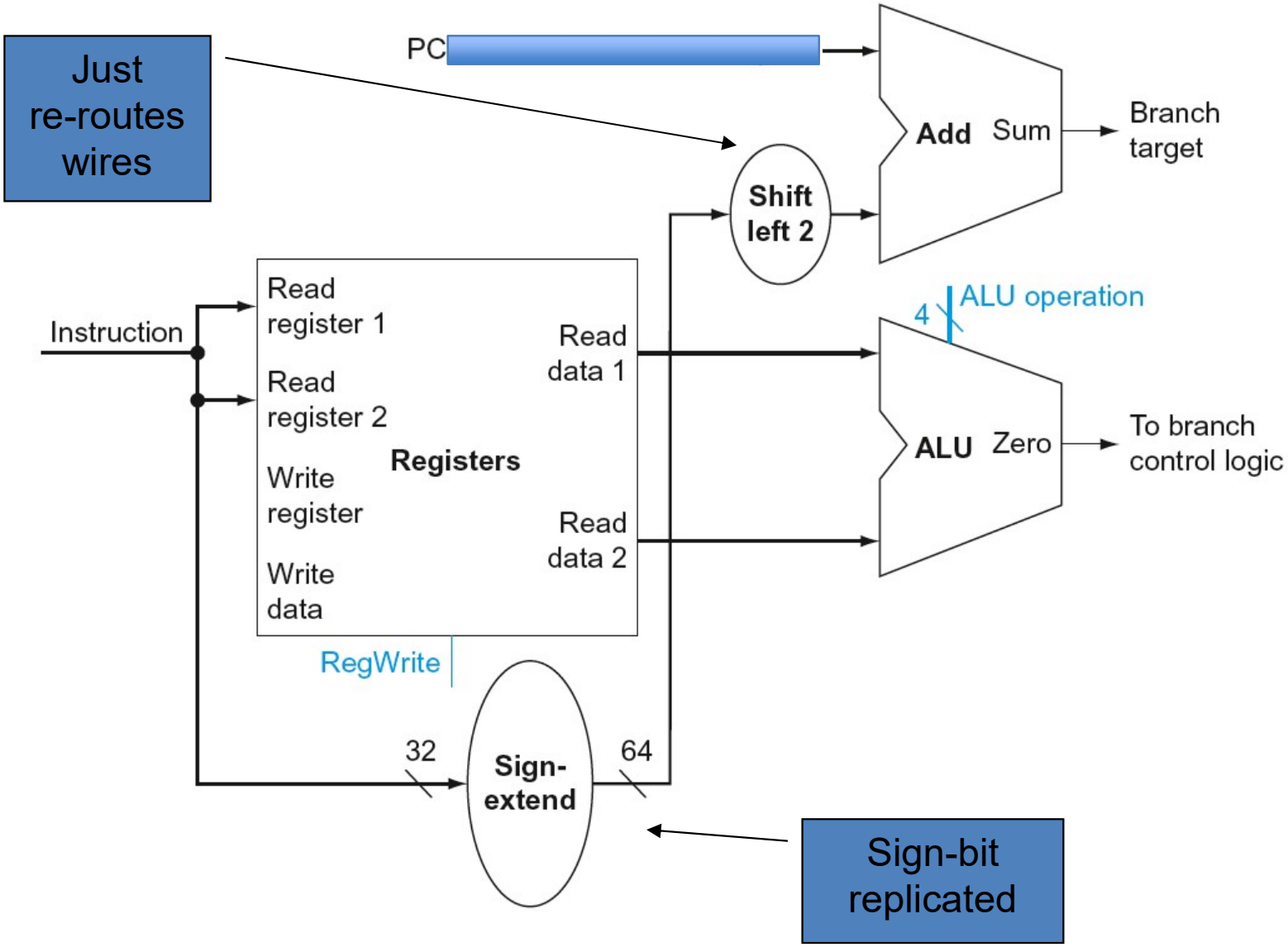

a. Data memory unit

b. Sign extension unit

# CBZ (Compare & Branch on Zero)

- Read register operand
- Pass it to ALU
  - Use ALU, check if zero
- Calculate target address
  - Sign-extend instruction (word) offset
  - Shift left 2 places (word to byte offset conversion)
  - Add to PC

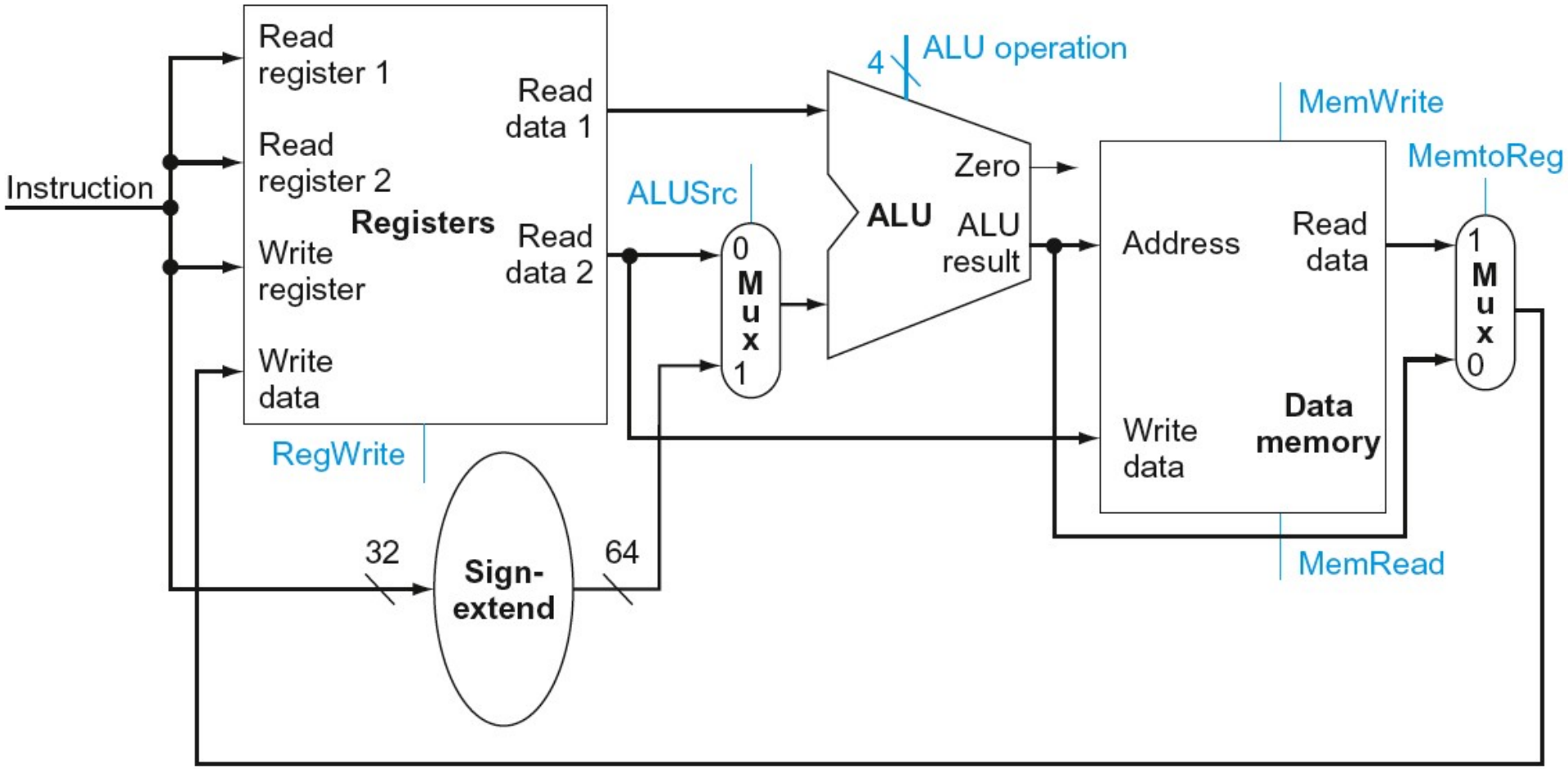
# Branch Instructions



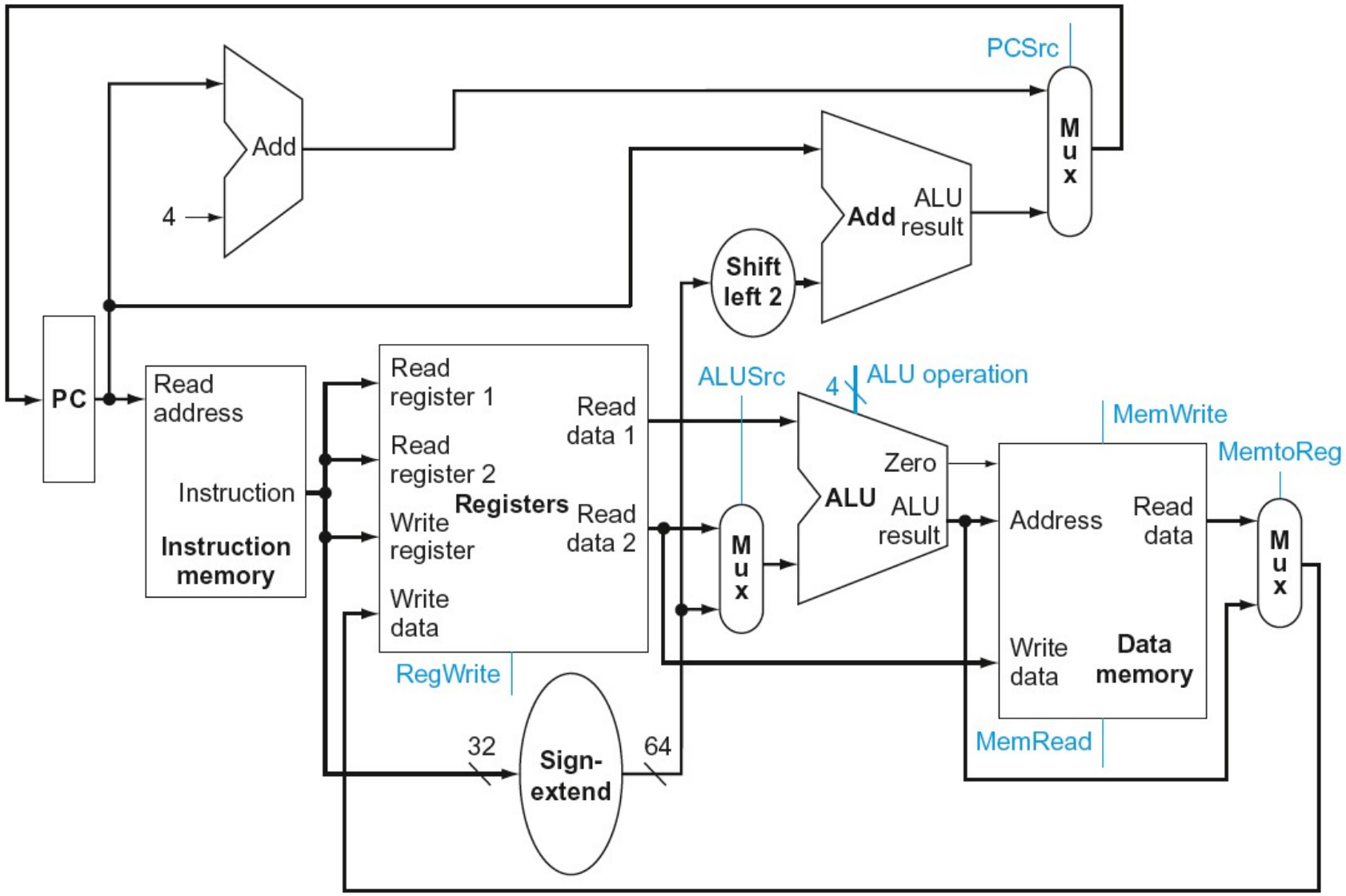
# Composing the Elements

- First-cut data path does an instruction in one clock cycle
  - Each datapath element can only do one function at a time
  - Hence, we need separate instruction and data memories
- Use multiplexers where alternate data sources are used for different instructions

# R-Type/Load/Store Datapath



# Full Datapath



# ALU Control



- ALU used for
  - Load/Store: F = add
  - CBZ: F = pass input b
  - R-type: F depends on opcode

| ALU control | Function |
|---|---|
| 0000 | AND |
| 0001 | OR |
| 0010 | add |
| 0110 | subtract |
| 0111 | Pass input b |
| 1100 | NOR |

# ALU Control

- Assume 2-bit ALUOp derived from opcode
  - Combinational logic derives ALU control

| Instruction | ALUOp | Instruction operation | Opcode field | Desired ALU action | ALU control input |
|---|---|---|---|---|---|
| LDUR | 00 | load register | XXXXXXXXXXX | add | 0010 |
| STUR | 00 | store register | XXXXXXXXXXX | add | 0010 |
| CBZ | 01 | compare and branch on zero | XXXXXXXXXXX | pass input b | 0111 |
| R-type | 10 | ADD | 10001011000 | add | 0010 |
| R-type | 10 | SUB | 11001011000 | subtract | 0110 |
| R-type | 10 | AND | 10001010000 | AND | 0000 |
| R-type | 10 | ORR | 10101010000 | OR | 0001 |

# The Main Control Unit

- Control signals derived from instruction by decoding.

| Field | opcode | Rm | shamt | Rn | Rd |
|---|---|---|---|---|---|
| Bit positions | 31:21 | 20:16 | 15:10 | 9:5 | 4:0 |

a. R-type instruction

| Field | 1986 or 1984 | address | 0 | Rn | Rt |
|---|---|---|---|---|---|
| Bit positions | 31:21 | 20:12 | 11:10 | 9:5 | 4:0 |

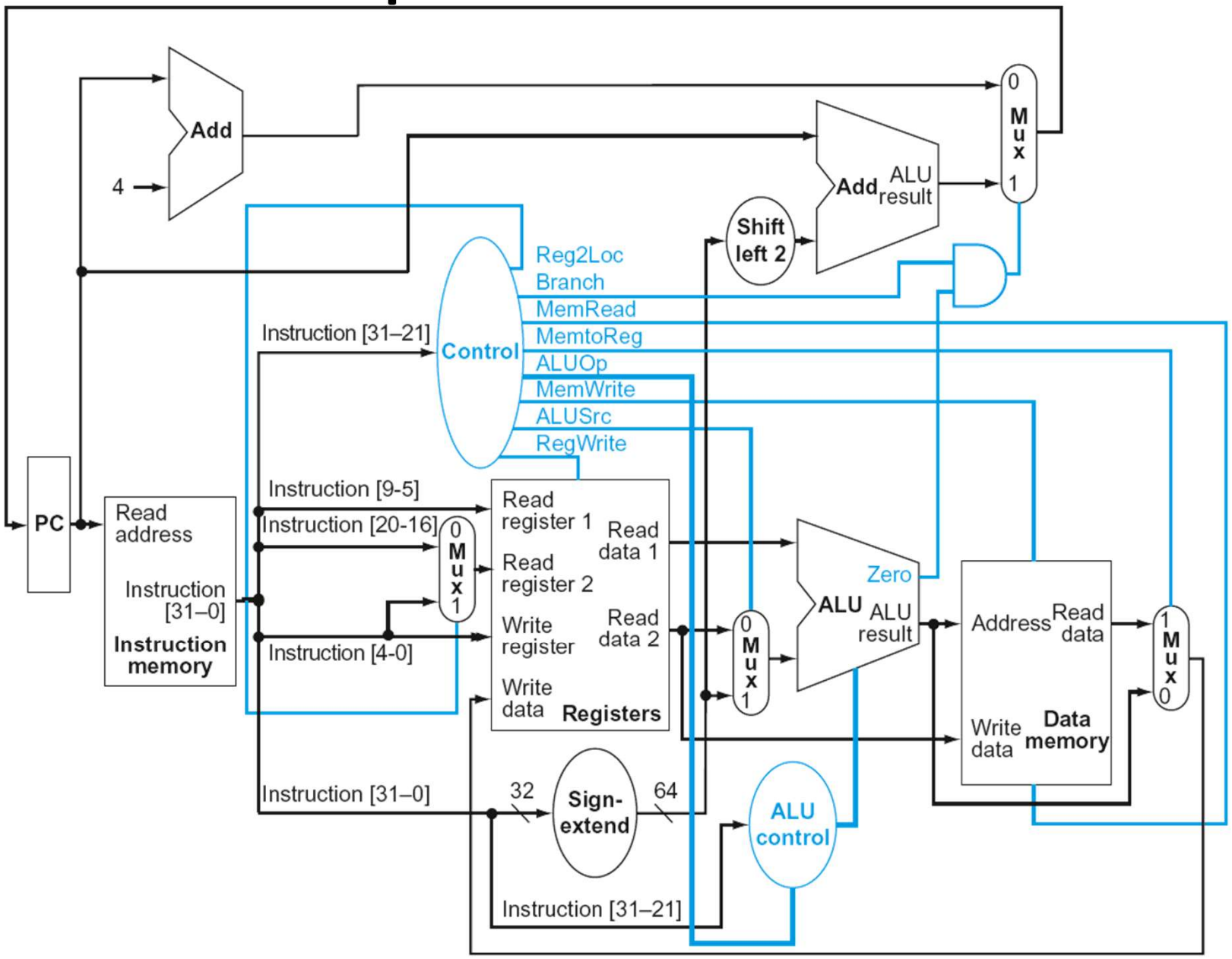
b. Load or store instruction

| Field | 180 | address | Rt |
|---|---|---|---|
| Bit positions | 31:26 | 23:5 | 4:0 |

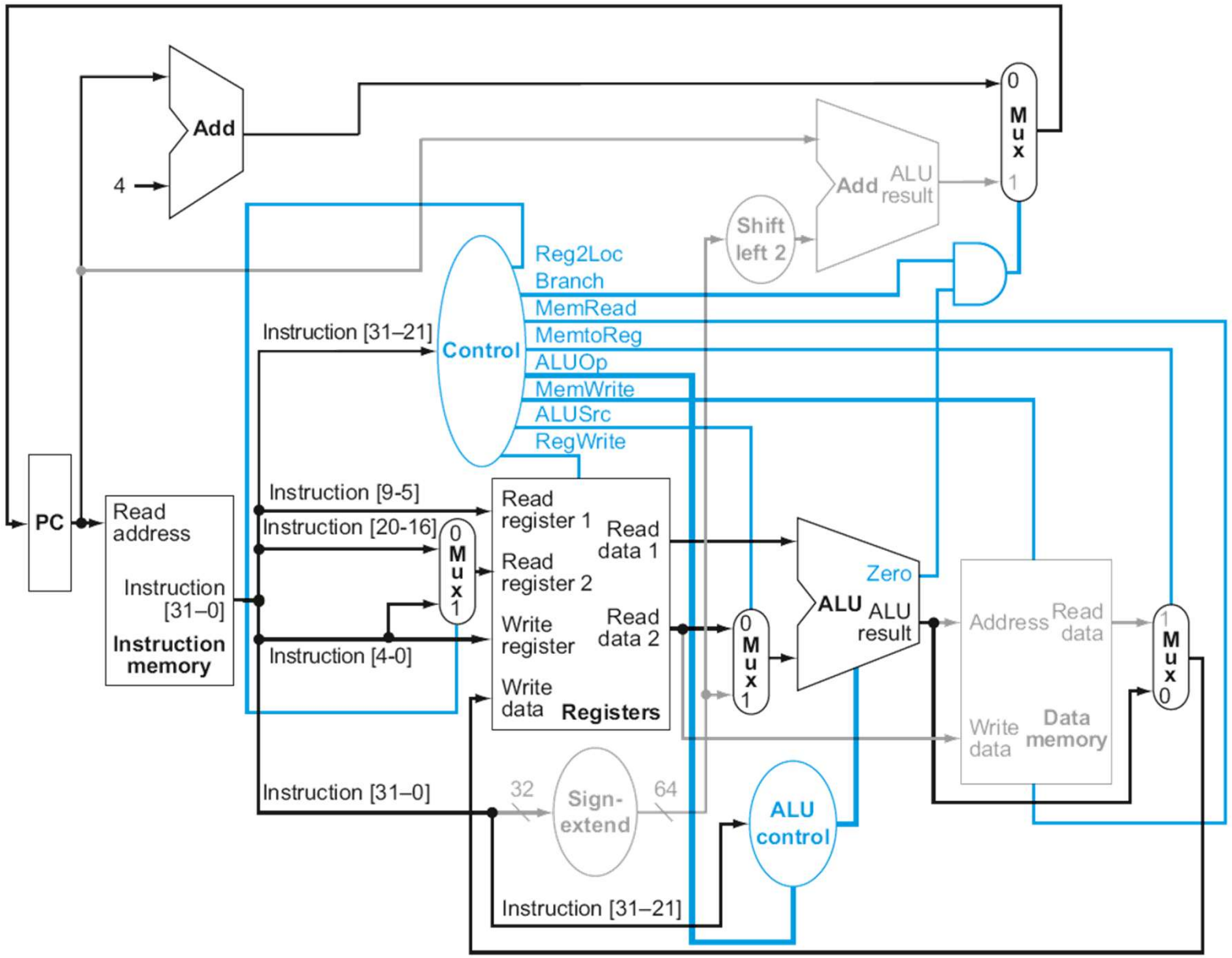
c. Conditional branch instruction

Typo, here.
Should be 31:24

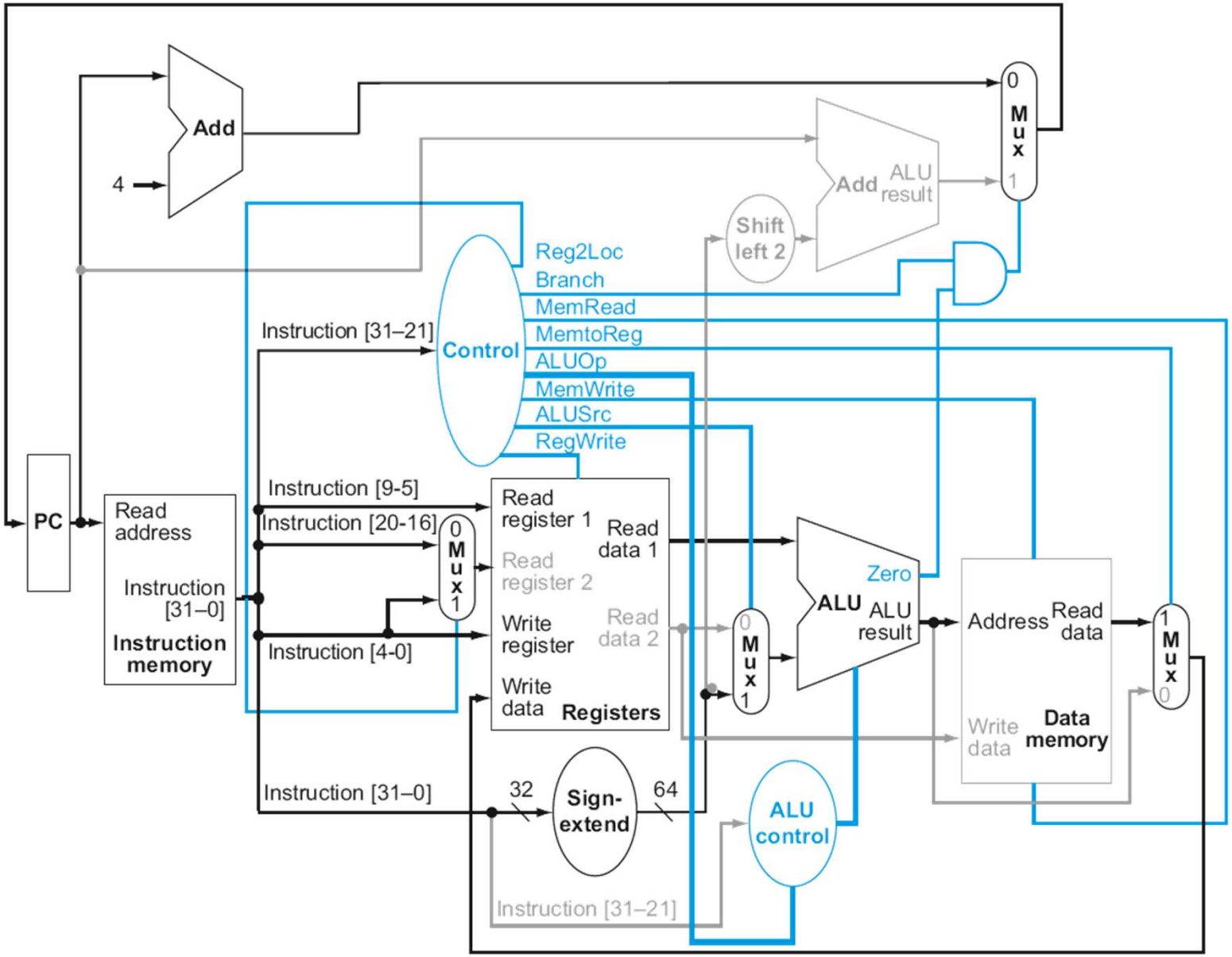
# Datapath With Control



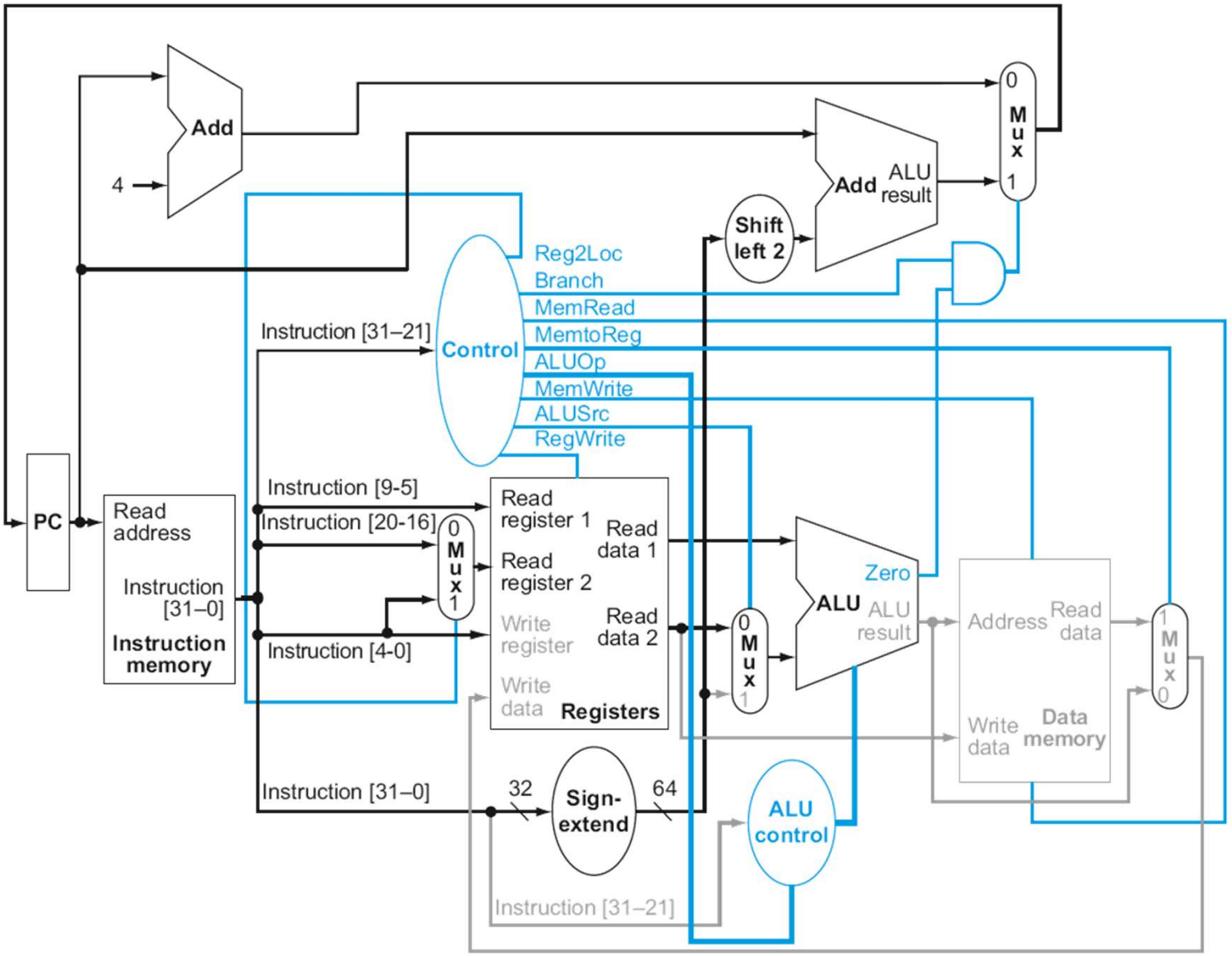
# R-Type Instruction



# Load Instruction



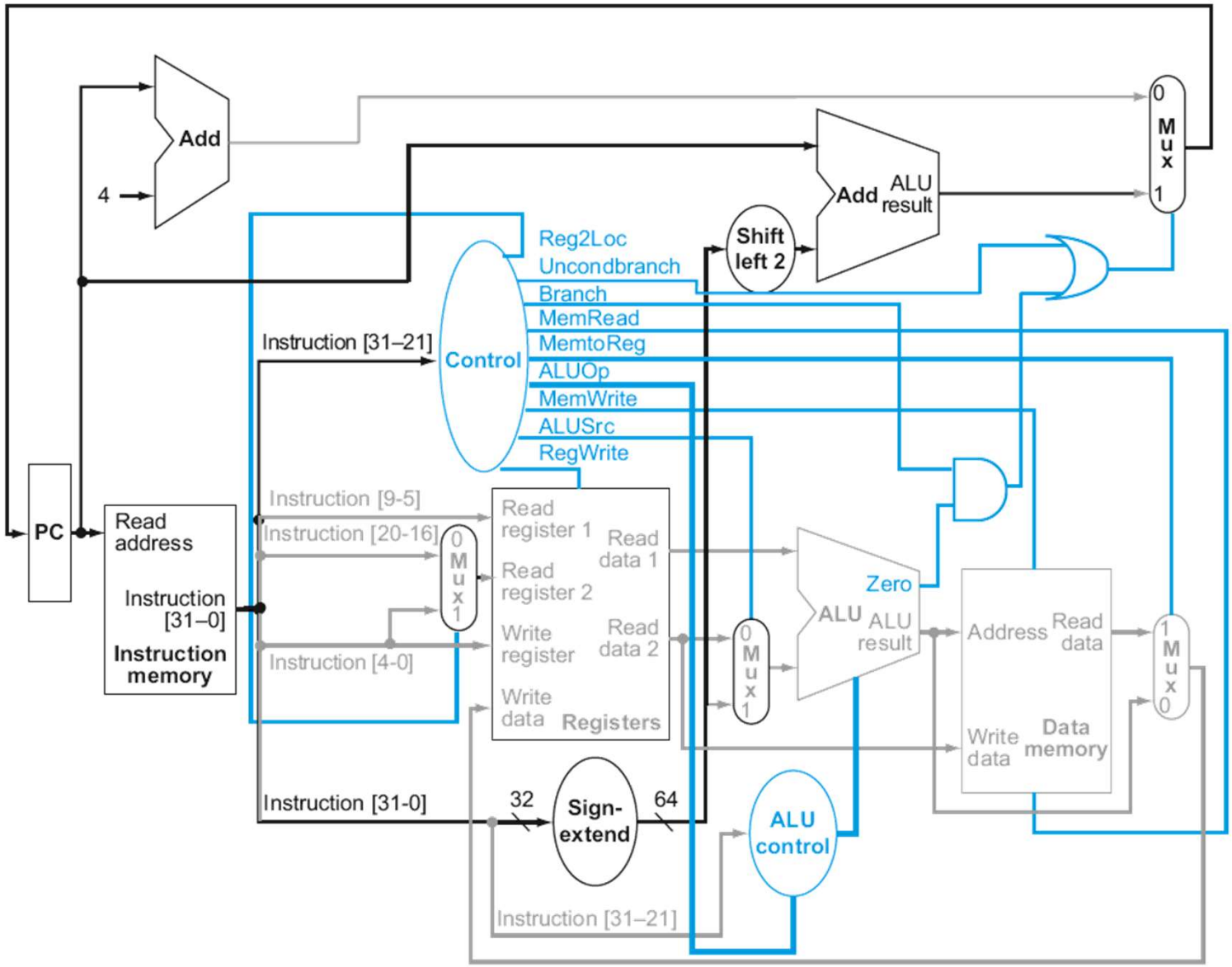
# CBZ Instruction



# Implementing Unconditional Branch (B)

- B-format: 6b opcode, 26b instruction offset
- Opcode: 0A0 to 0BF = 000 101X XXXX – first 6b in common.
- Note offset is given as instruction offset. So, we need to convert it to byte offset by shifting 2b to left. Also, sign-extend it to 64b.
- PC = PC + byte offset
- Need an extra control signal (Uncondbranch) decoded from opcode

# Datapath With B Added

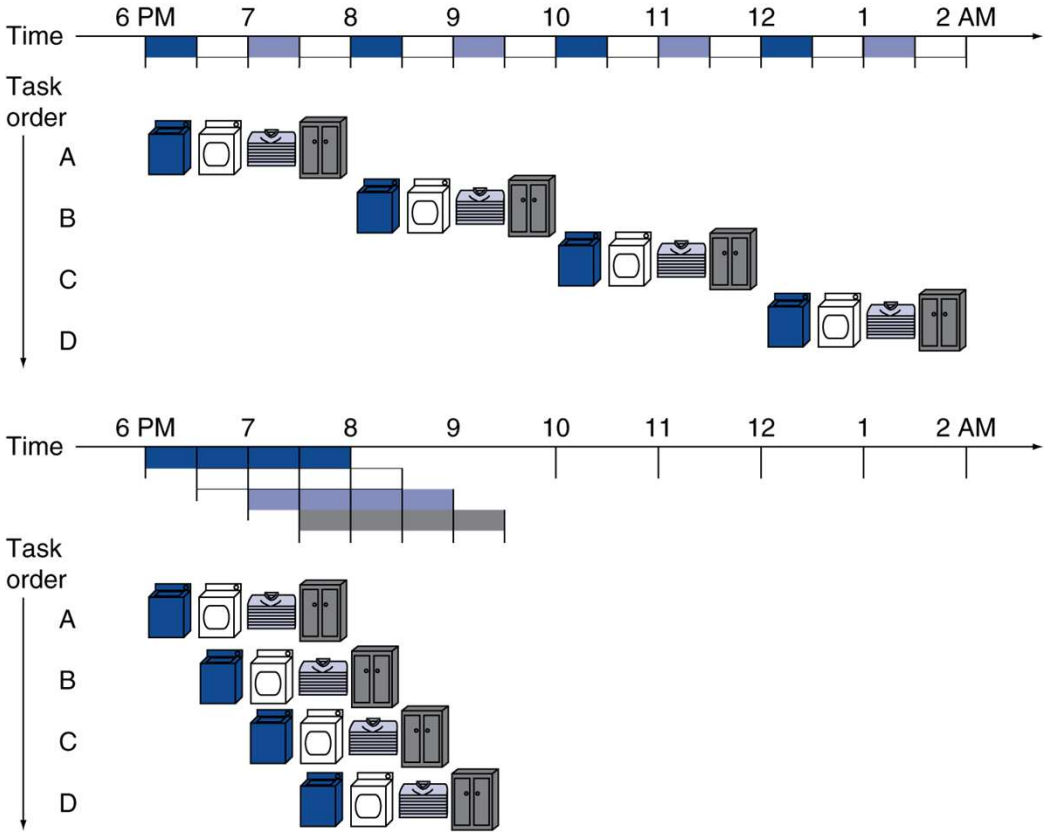


# Performance Issues

- Longest delay determines clock period
  - Critical path: load instruction
  - Instruction memory → register file → ALU → data memory → register file
- Not feasible to vary period for different instructions
- Violates design principle
  - Making the common case fast
- We will improve performance by pipelining

# Pipelining Analogy

- Pipelined laundry: overlapping execution
  - Parallelism improves performance




- Four loads:
  - Speedup = 8/3.5 = 2.3
- Non-stop:
  - Speedup = 2n/0.5n + 1.5 ≈ 4 = number of stages

# LEGv8 Pipeline

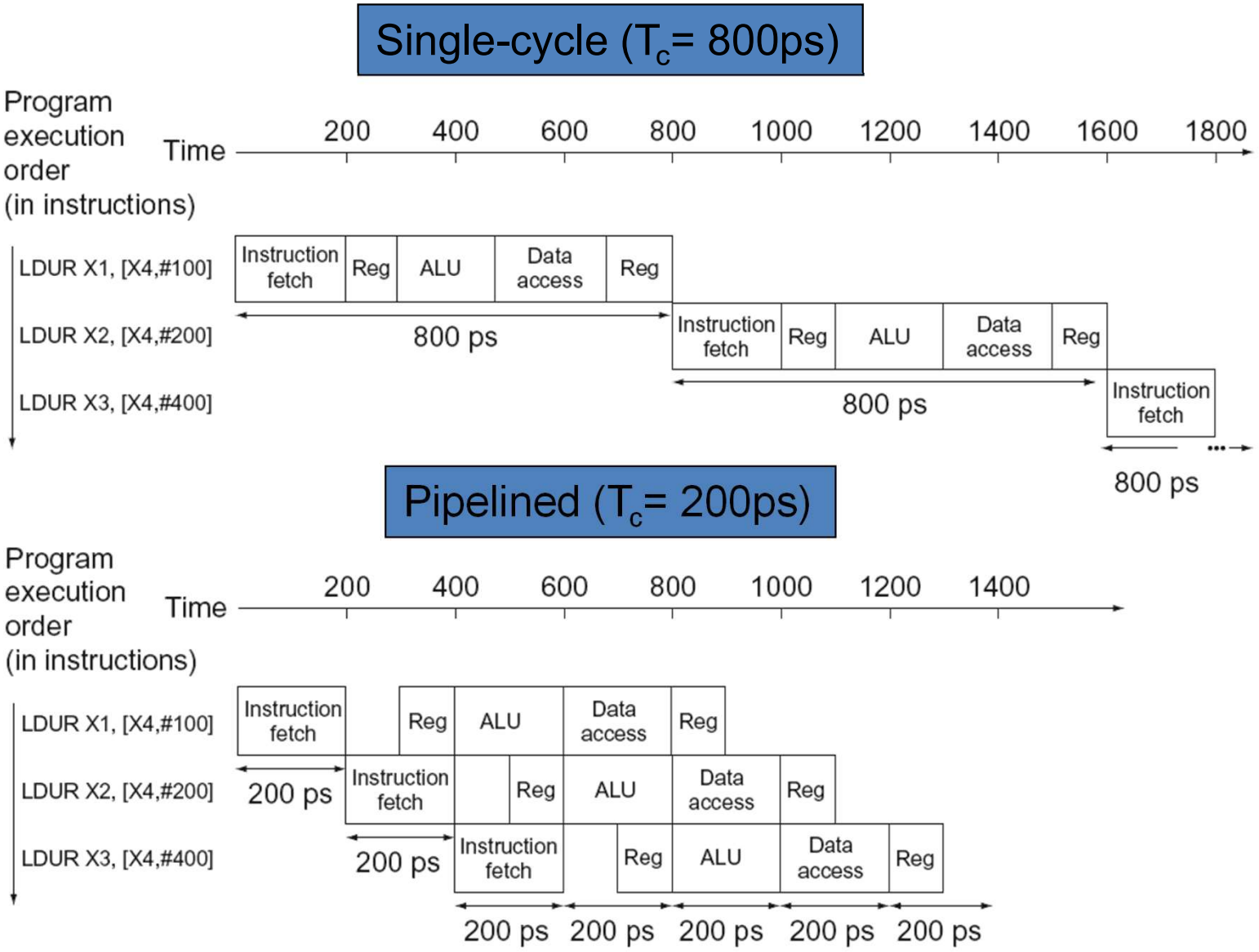
- Five stages, one step per stage
  1. IF: Instruction fetch from memory
  2. ID: Instruction decode & register read
  3. EX: Execute operation or calculate address
  4. MEM: Access memory operand
  5. WB: Write result back to register

# Pipeline Performance

- Assume time for stages is
  - 100ps for register read or write
  - 200ps for other stages
  - RF write & read done in 1 clock cycle
- Compare pipelined datapath with single-cycle datapath

| Instr | Instr fetch | Register read | ALU op | Memory access | Register write | Total time |
|---|---|---|---|---|---|---|
| LDUR | 200ps | 100 ps | 200ps | 200ps | 100 ps | 800ps |
| STUR | 200ps | 100 ps | 200ps | 200ps | | 700ps |
| R-format | 200ps | 100 ps | 200ps | | 100 ps | 600ps |
| CBZ | 200ps | 100 ps | 200ps | | | 500ps |

# Pipeline Performance



# Pipeline Speedup

- If all stages are balanced
  - i.e., all take the same time
  - Time between instructions$_{pipelined}$ = $\frac{\text{Time between instructions}_{nonpipelined}}{\text{Number of stages}}$
- If not balanced, speedup is less
- Speedup due to increased throughput
  - Latency (time for each instruction) does not decrease

# Pipelining and ISA Design

- LEGv8 ISA designed for pipelining
  - All instructions are 32-bits
    - Easier to fetch and decode in one cycle
    - c.f. x86: 1- to 17-byte instructions
  - Few and regular instruction formats
    - Can decode and read registers in one step
  - Load/store addressing
    - Can calculate address in 3rd stage, access memory in 4th stage
  - Alignment of memory operands
    - Memory access takes only one cycle

# Hazards

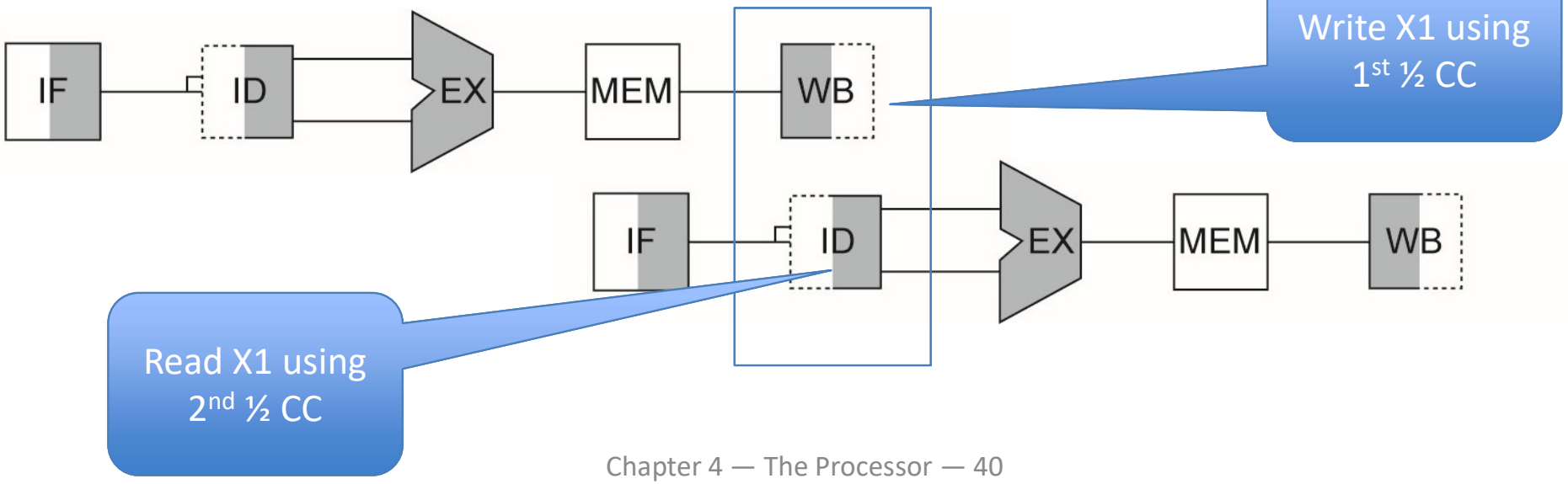
- Situations that prevent starting the next instruction in the next cycle
- Structure hazards
  - A required resource is busy
- Data hazard
  - Need to wait for previous instruction to complete its data read/write
- Control hazard
  - Deciding on control action depends on previous instruction

# Structure Hazards

- Conflict for use of a resource
- In LEGv8 pipeline with a single memory
  - Load/store requires data access
  - Instruction fetch would have to *stall* for that cycle
    - Would cause a pipeline “bubble”
- Hence, pipelined datapaths require separate instruction/data memories
  - Or separate instruction/data caches

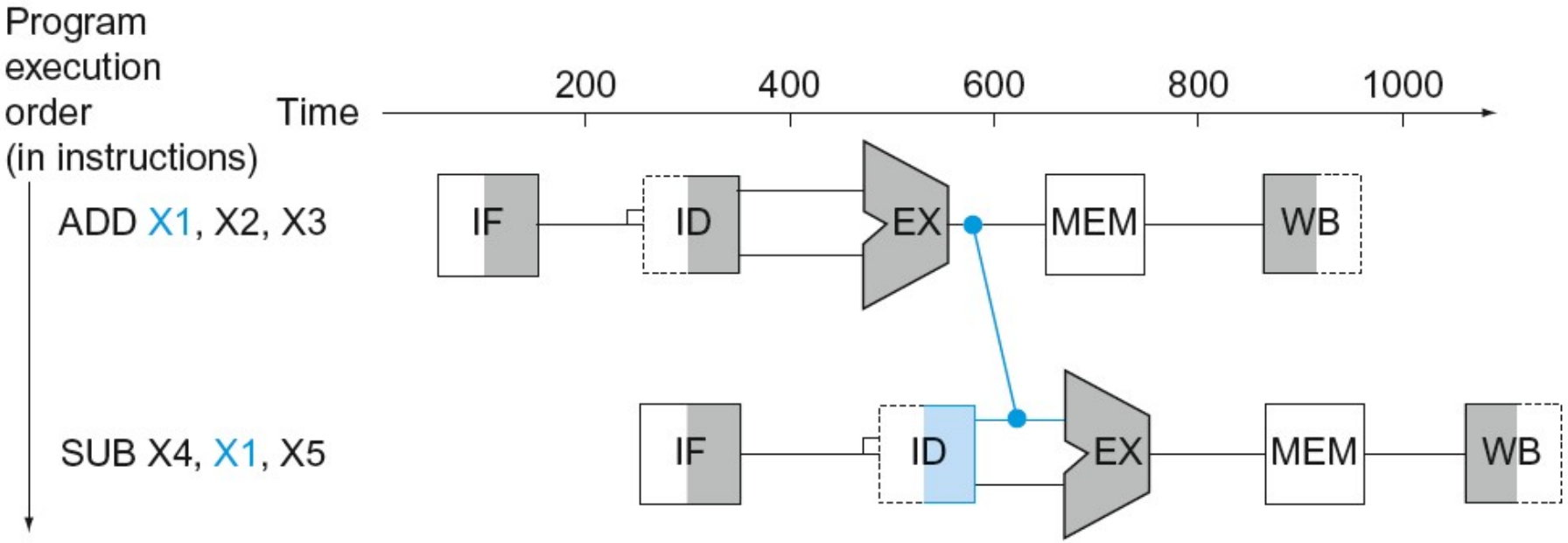
# Data Hazards

- An instruction depends on completion of data access by a previous instruction
  - ADD X1, X2, X3
    SUB X4, X1, X5
- How many stall (bubble) clock cycles are needed? Two.



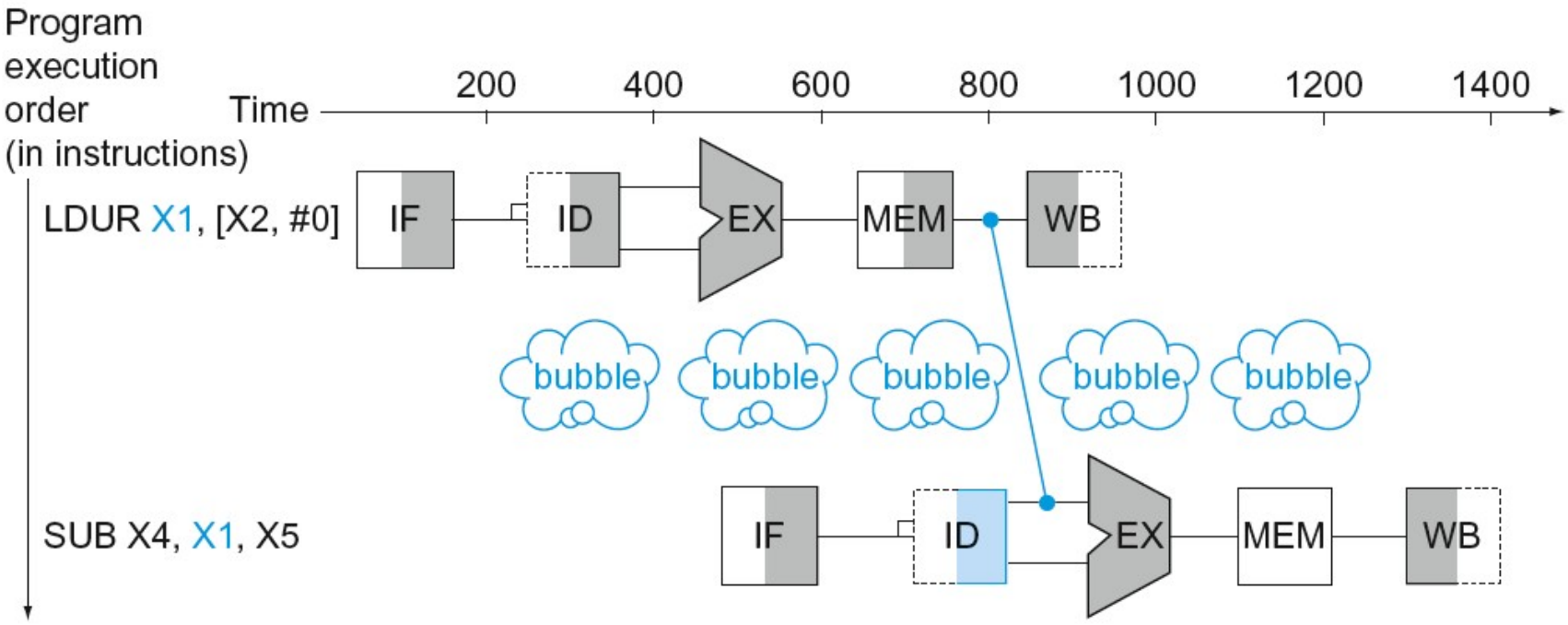
# Forwarding (aka Bypassing)

- Use result when it is computed
  - Don't wait for it to be stored in a register
  - Requires extra connections in the datapath
  - EXE-forwarding shown below – no stall needed



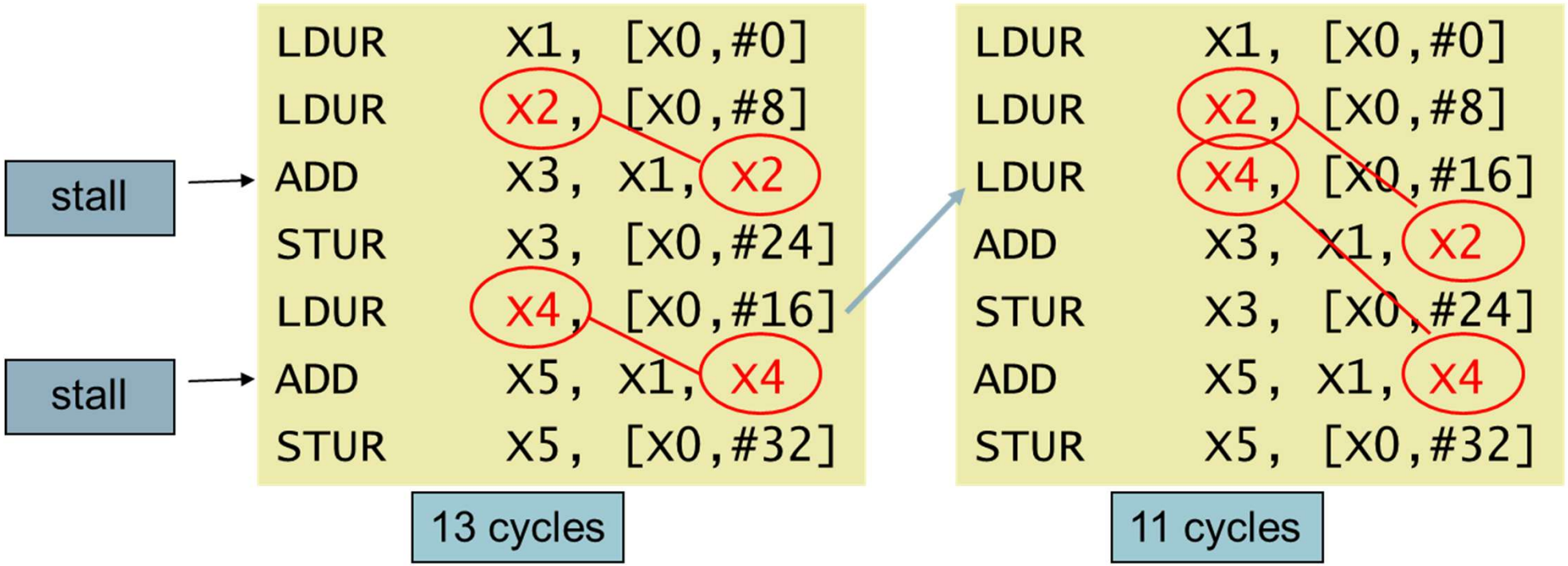
# Load-Use Data Hazard

- Can't always avoid stalls by forwarding
  - If value not computed when needed
  - Can't forward backward in time!
  - MEM-forwarding shown below – 1 stall CC needed



# Code Scheduling to Avoid Stalls

- Reorder code to avoid use of load result in the next instruction
- C code for `A = B + E; C = B + F;`

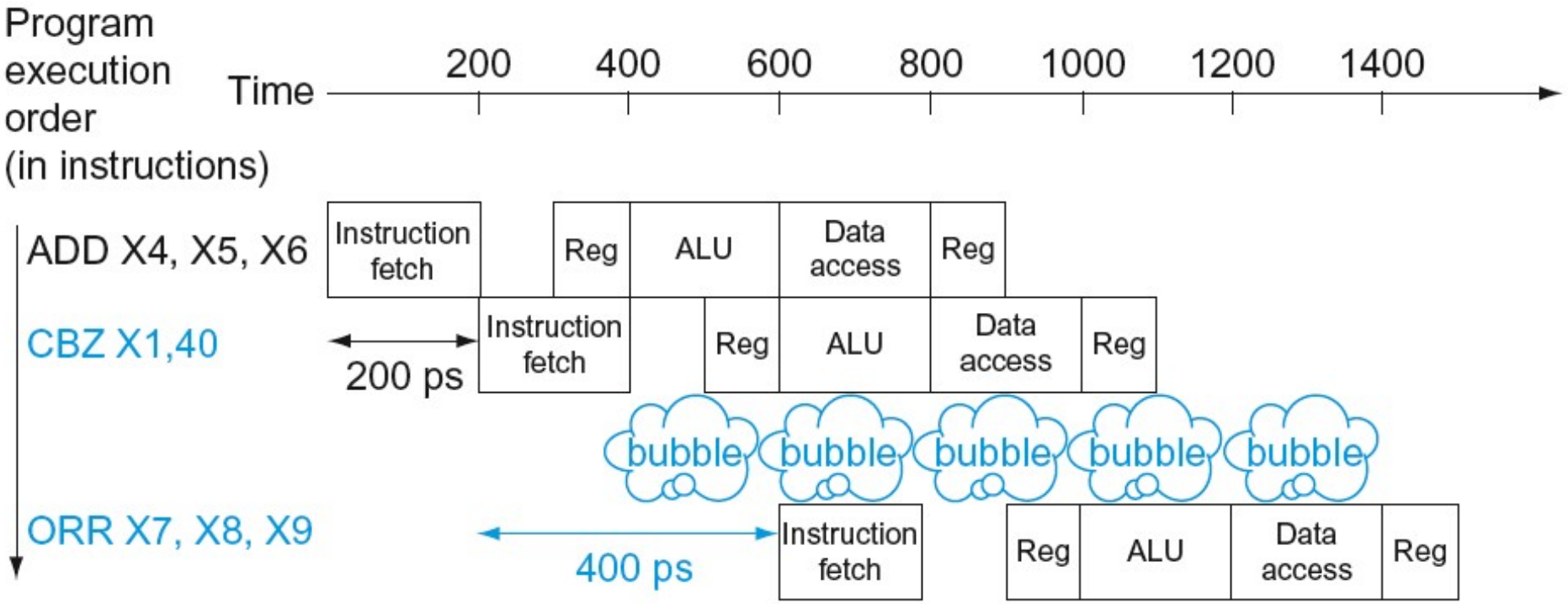


# Control Hazards

- Branch determines flow of control
  - Fetching next instruction depends on branch outcome
  - Pipeline can't always fetch correct instruction
    - Still working on ID stage of branch
- In LEGv8 pipeline
  - Need to compare registers and compute target early in the pipeline
  - Can add hardware to do it in ID stage

# Stall on Branch

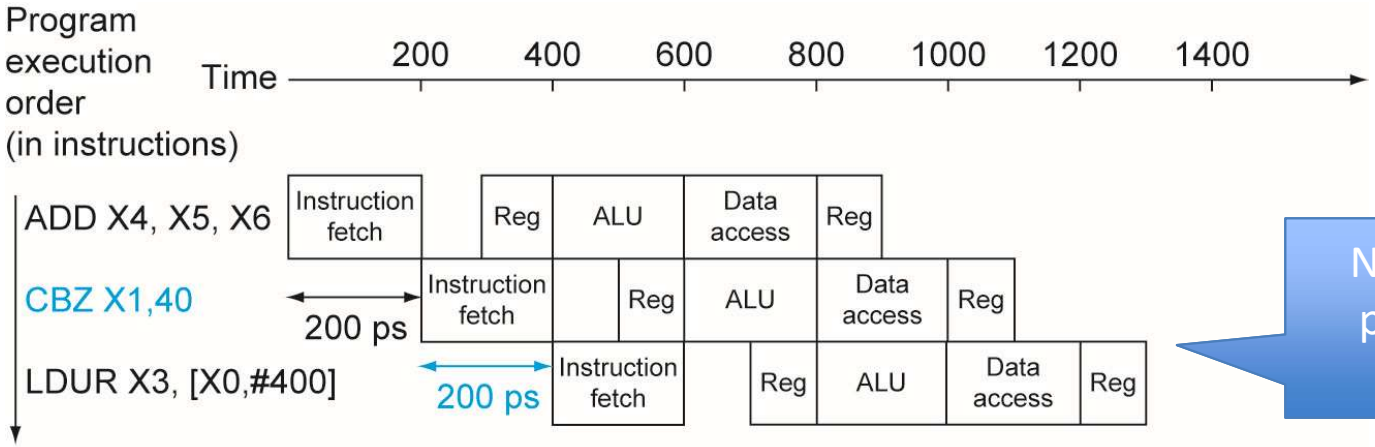
- Assume branch decision made in ID stage
- Wait until branch outcome determined before fetching next instruction – one stall needed

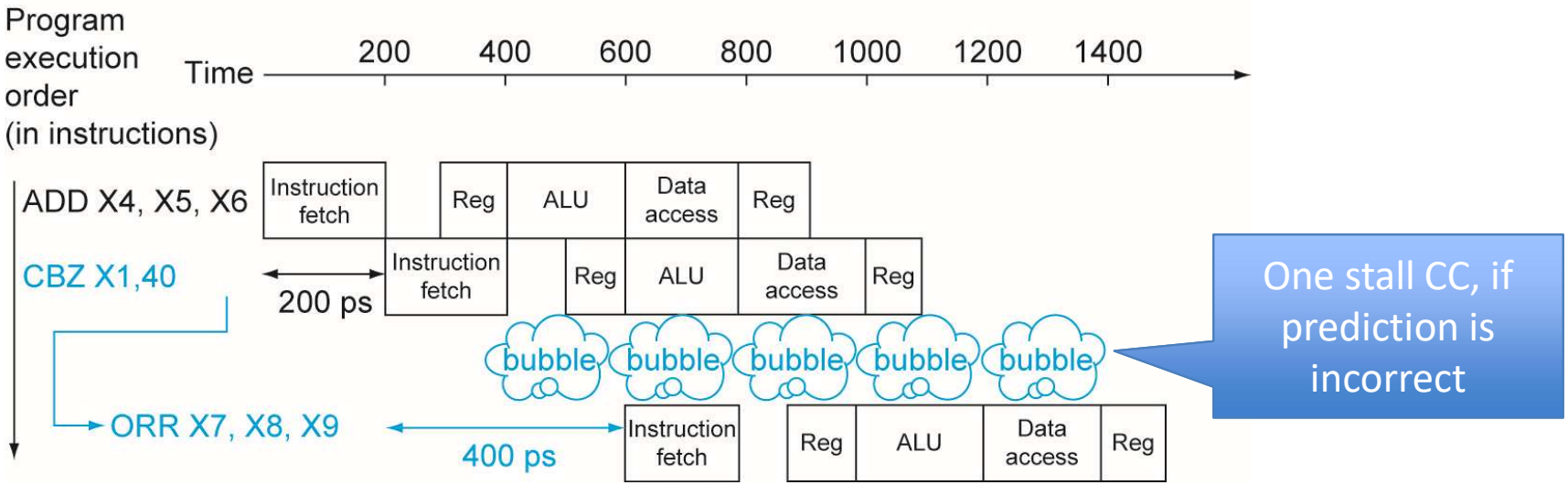


# Branch Prediction

- Longer pipelines can't readily determine branch outcome early
  - Stall penalty becomes unacceptable
- Predict outcome of branch
  - Only stall if prediction is wrong
- In LEGv8 pipeline
  - Can predict branches not taken
  - Fetch instruction after branch, with no delay

# Predict Not-Taken Example



# More-Realistic Branch Prediction

- Static branch prediction
  - Based on typical branch behavior
  - Example: loop and if-statement branches
    - Predict backward branches taken - loop
    - Predict forward branches not taken – if-statement
- Dynamic branch prediction
  - Hardware measures actual branch behavior
    - e.g., record recent history of each branch
  - Assume future behavior will continue the trend
    - When wrong, stall while re-fetching, and update history

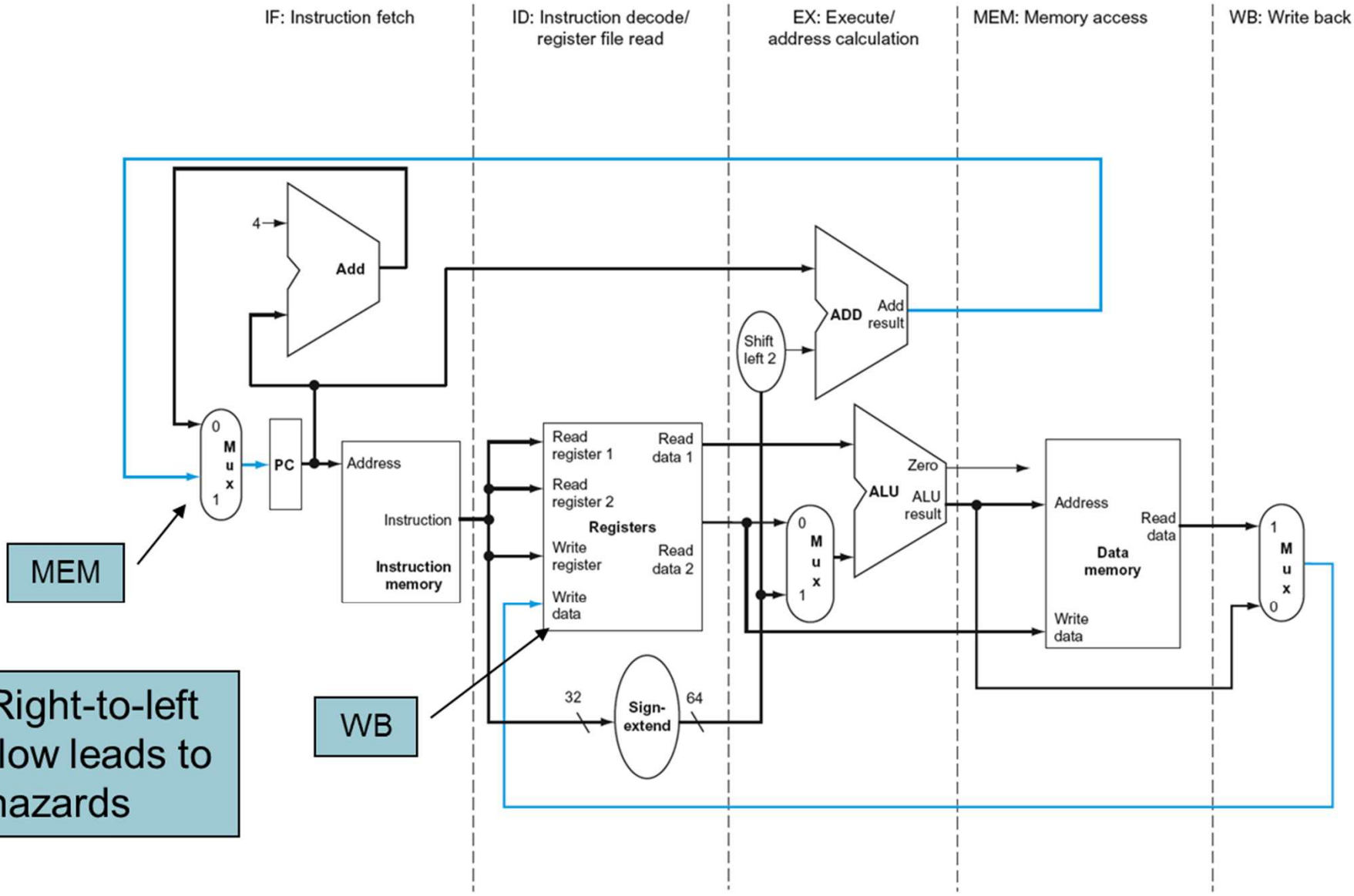
# Pipeline Summary

**The BIG Picture**

- Pipelining improves performance by increasing instruction throughput
  - Executes multiple instructions in parallel
  - Each instruction has the same latency
- Subject to hazards
  - Structure, data, control
- Instruction set design affects complexity of pipeline implementation

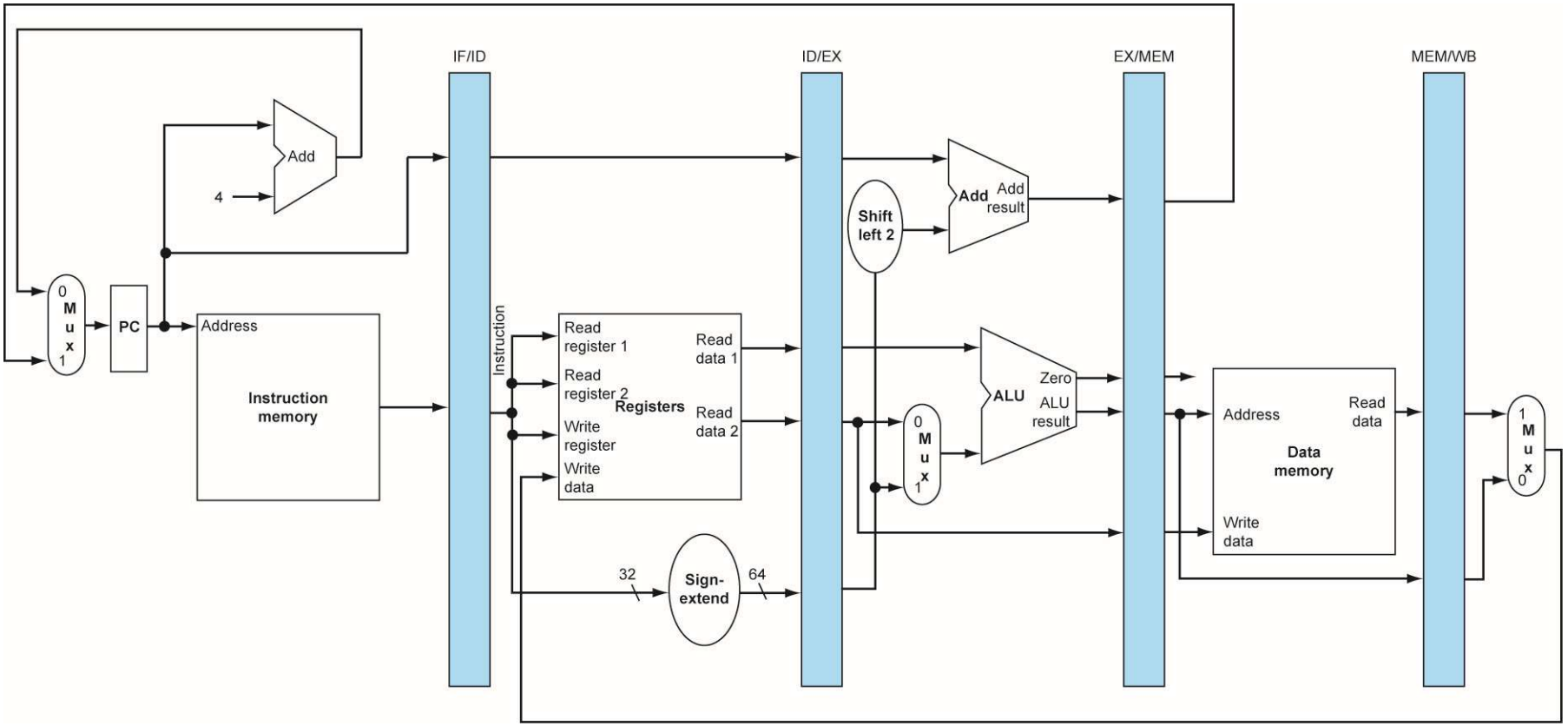
# LEGv8 Pipelined Datapath





# Pipeline registers

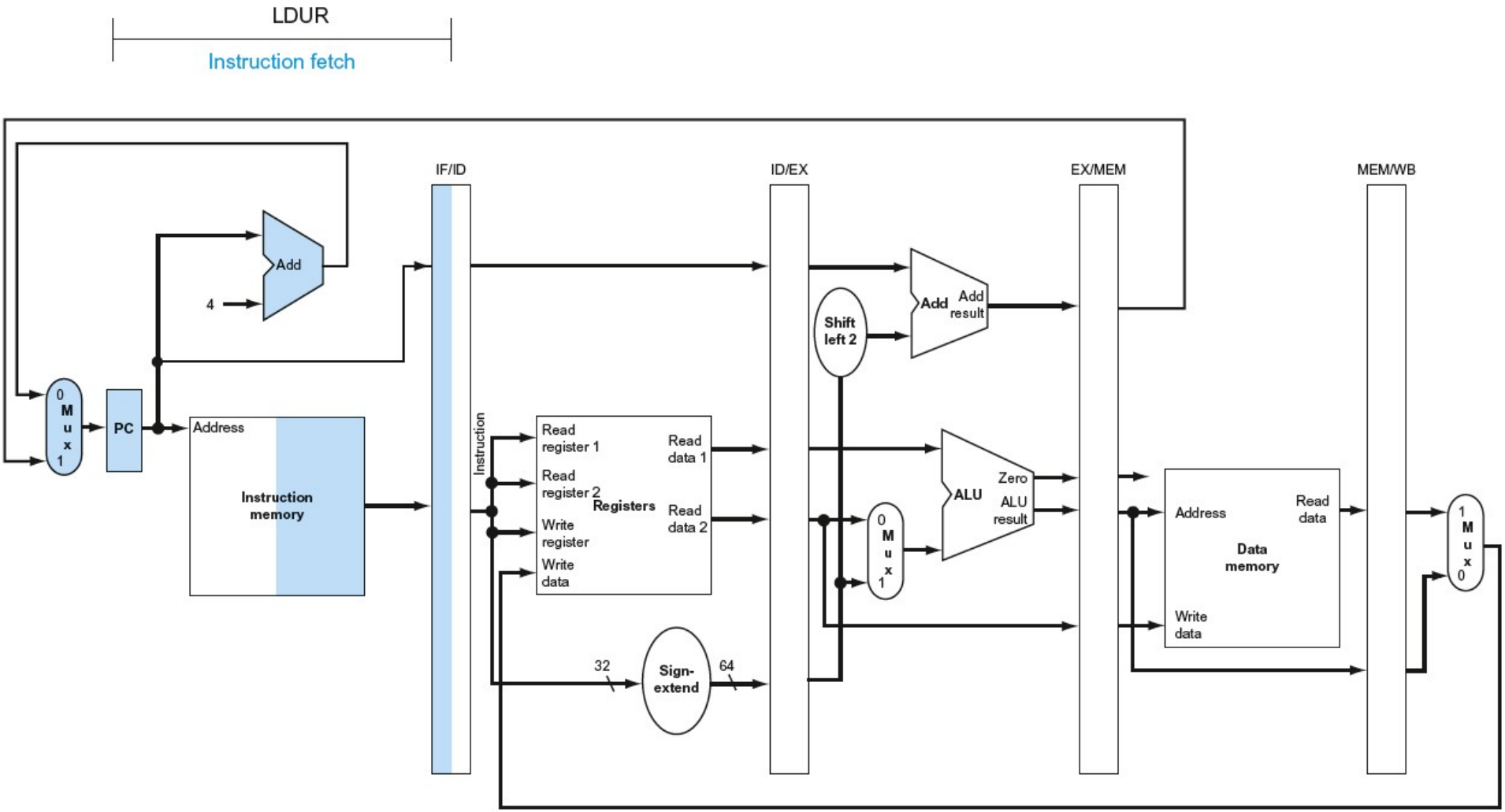
- Need registers between stages
  - To hold information produced in previous cycle



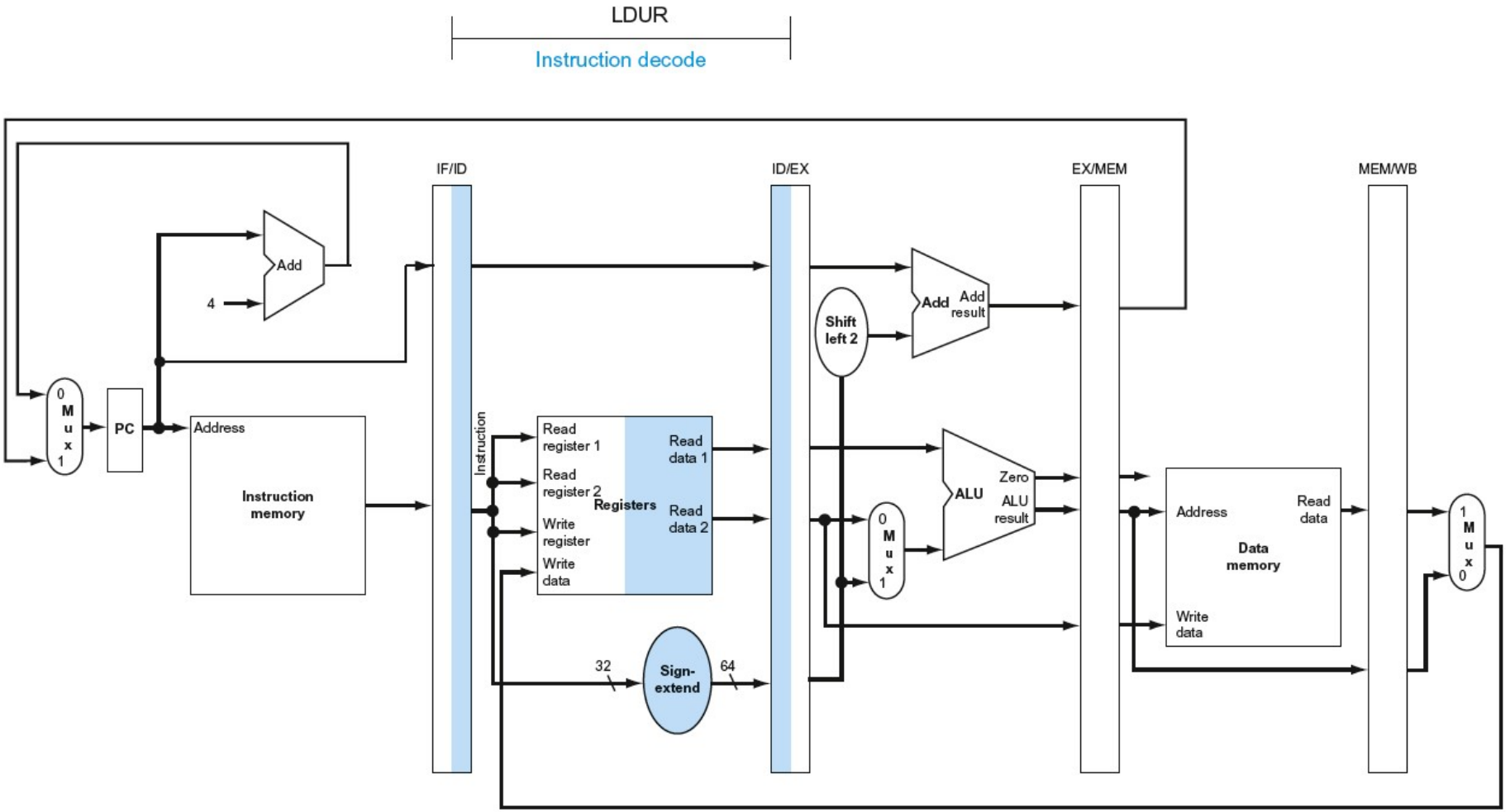
# Pipeline Operation

- Cycle-by-cycle flow of instructions through the pipelined datapath
  - "Single-clock-cycle" pipeline diagram
    - Shows pipeline usage in a single cycle
    - Highlight resources used
  - c.f. "multi-clock-cycle" diagram
    - Graph of operation over time
- We'll look at "single-clock-cycle" diagrams for load & store

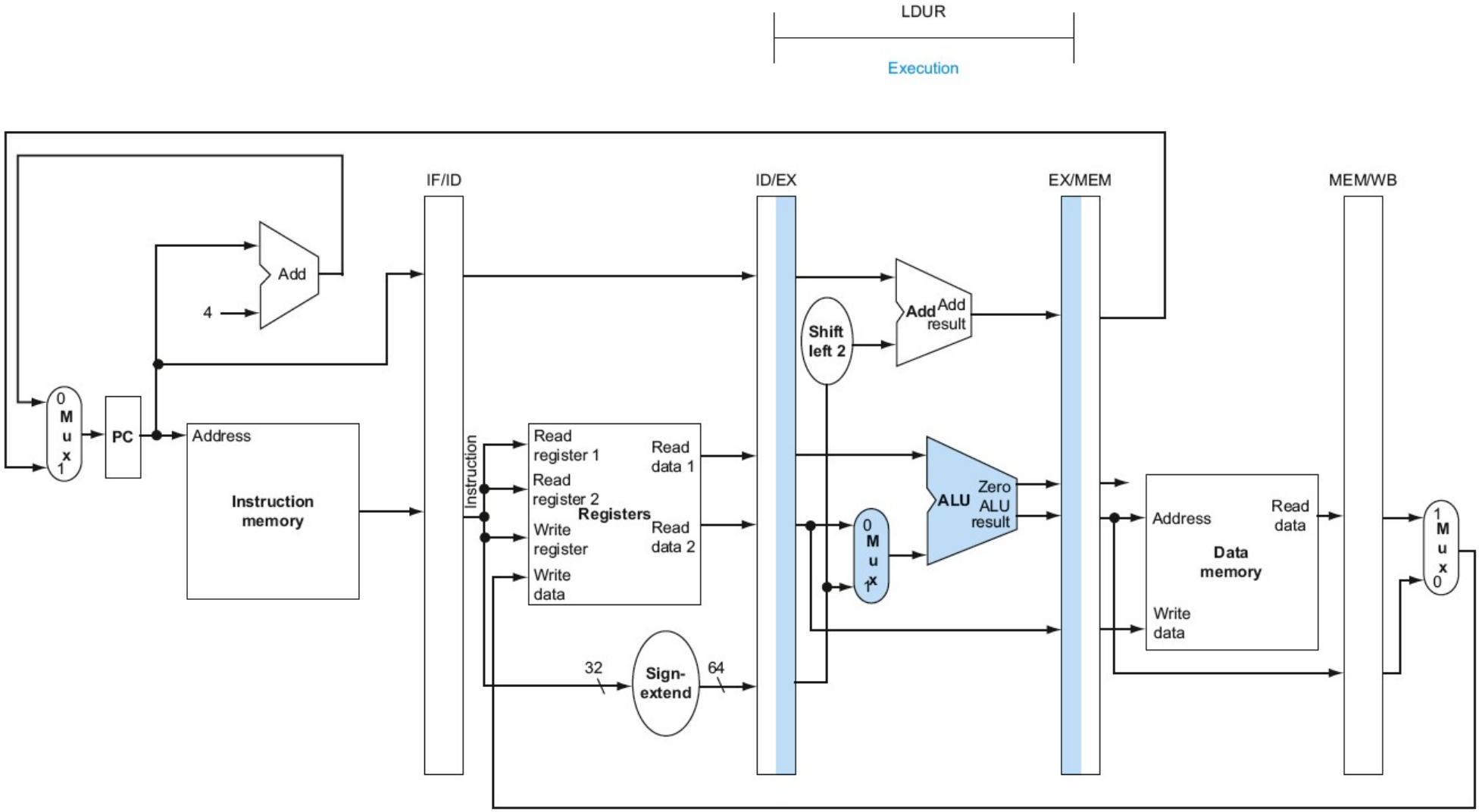
# IF for Load, Store, …



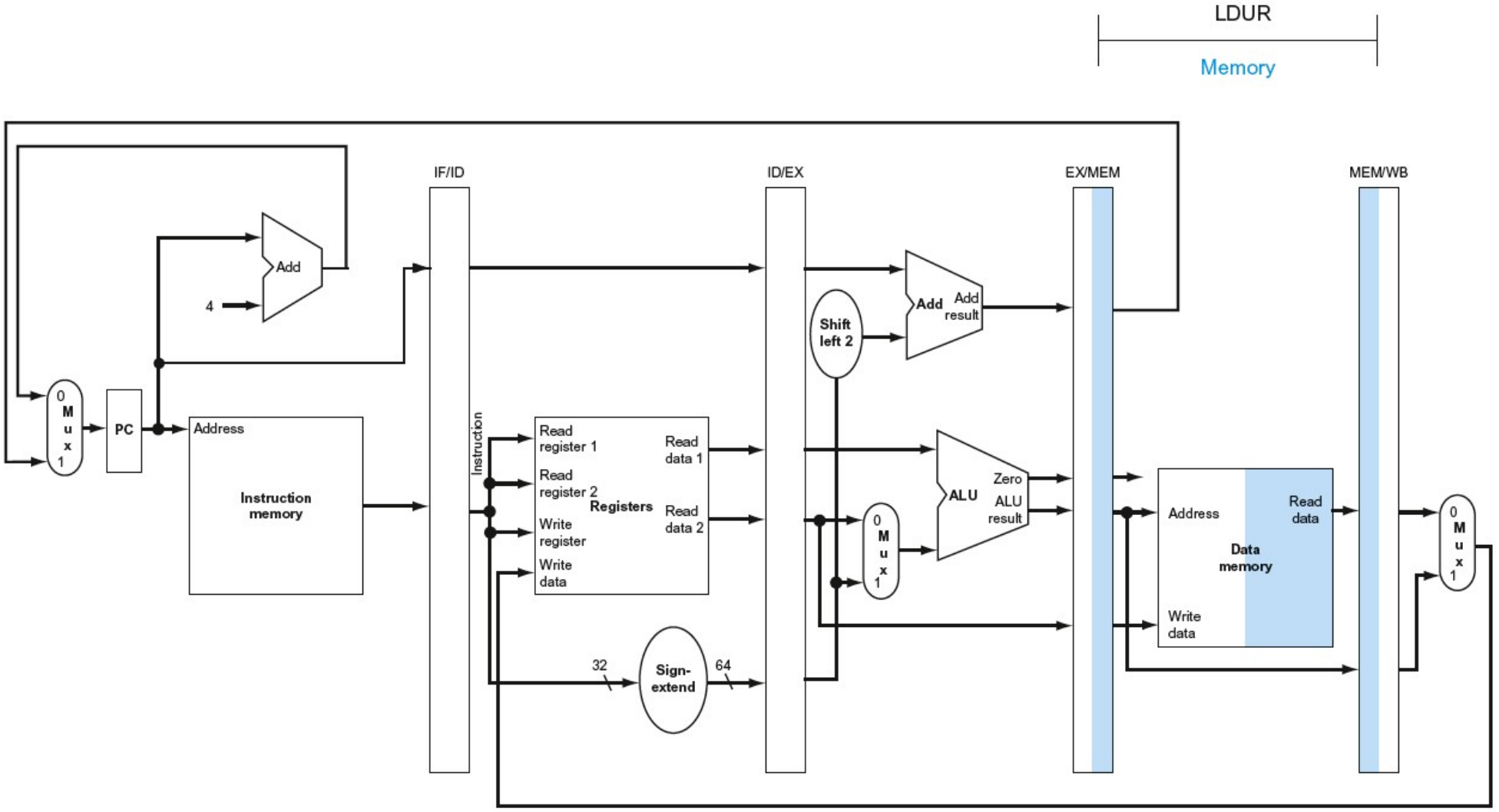
# ID for Load, Store, …



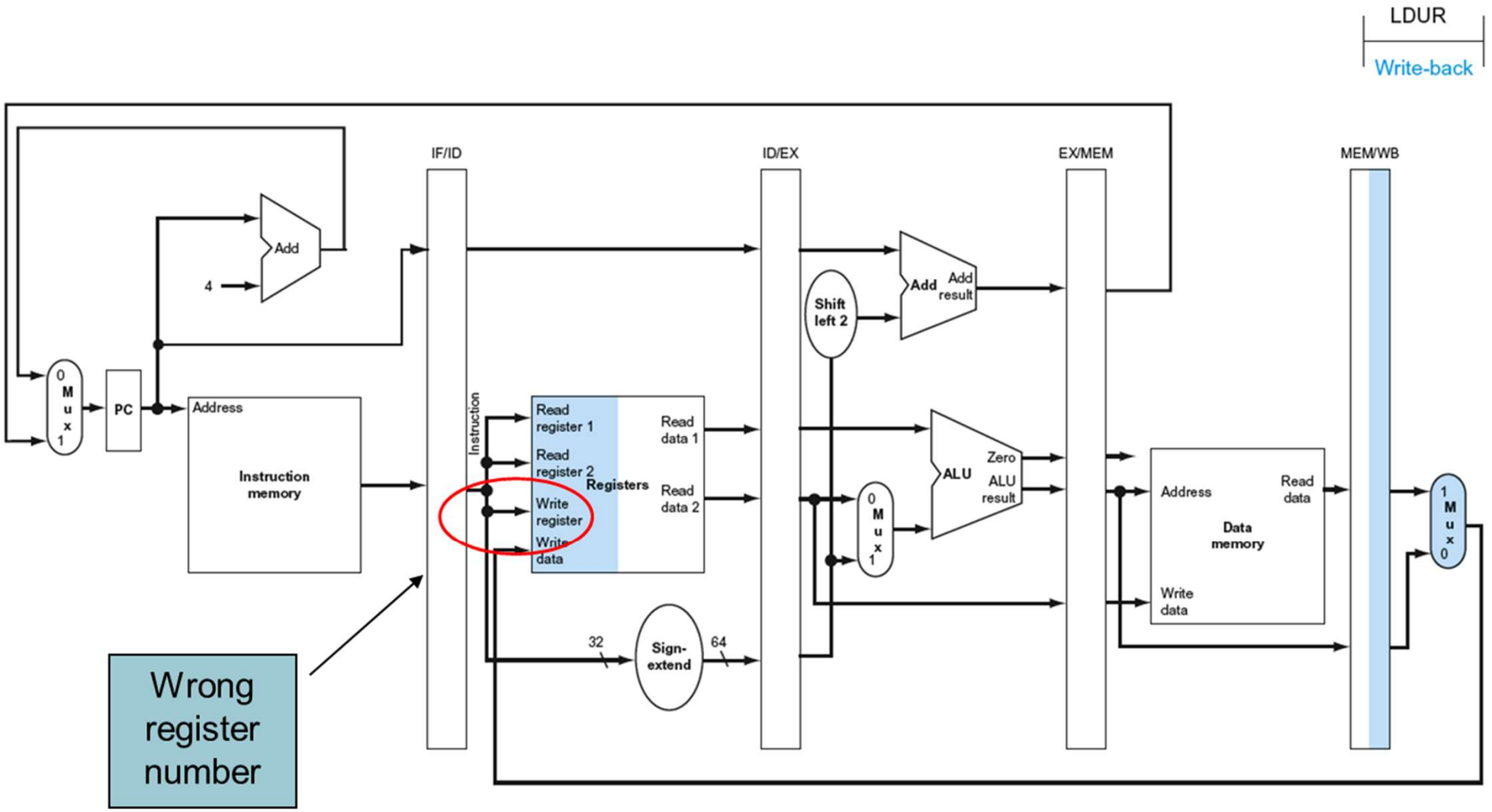
# EX for Load


LDUR
Execution
IF/ID
ID/EX
EX/MEM
MEM/WB
Add
4
M u x
0
1
PC
Address
Instruction memory
Instruction
Read register 1
Read register 2
Registers
Write register
Write data
Read data 1
Read data 2
Shift left 2
Add
Add result
ALU
Zero
ALU result
Address
Read data
Data memory
Write data
32
Sign-extend
64

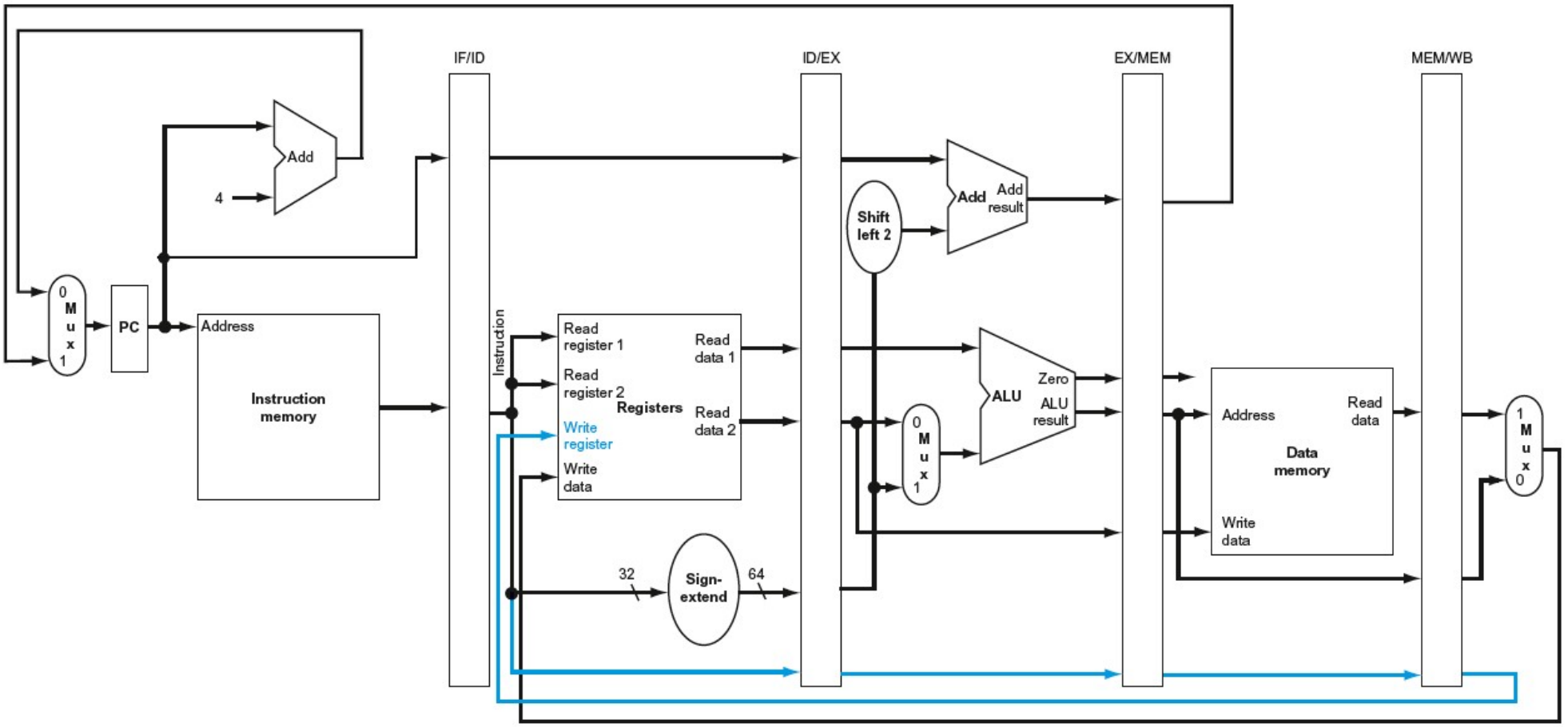
# MEM for Load


LDUR
Memory
IF/ID
ID/EX
EX/MEM
MEM/WB
Add
4
M u x
0
1
PC
Address
Instruction memory
Instruction
Read register 1
Read register 2
Registers
Write register
Write data
Read data 1
Read data 2
Shift left 2
Add
Add result
ALU
Zero
ALU result
Address
Read data
Data memory
Write data
32
Sign-extend
64

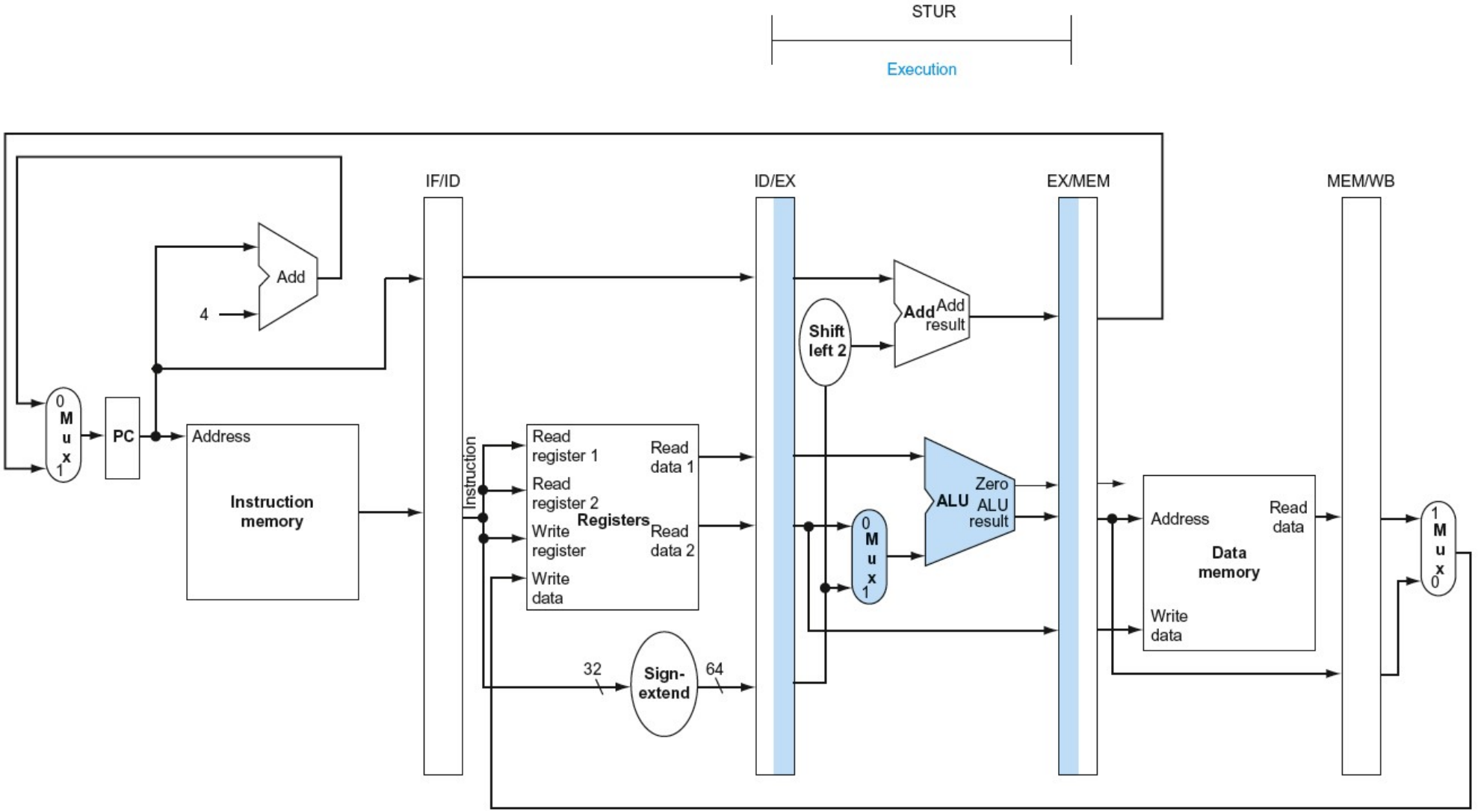
# WB for Load


LDUR
Write-back
IF/ID
ID/EX
EX/MEM
MEM/WB
Add
4
M u x
0
1
PC
Address
Instruction memory
Instruction
Read register 1
Read register 2
Registers
Write register
Write data
Read data 1
Read data 2
Shift left 2
Add
Add result
ALU
Zero
ALU result
Address
Read data
Data memory
Write data
32
Sign-extend
64
Wrong register number

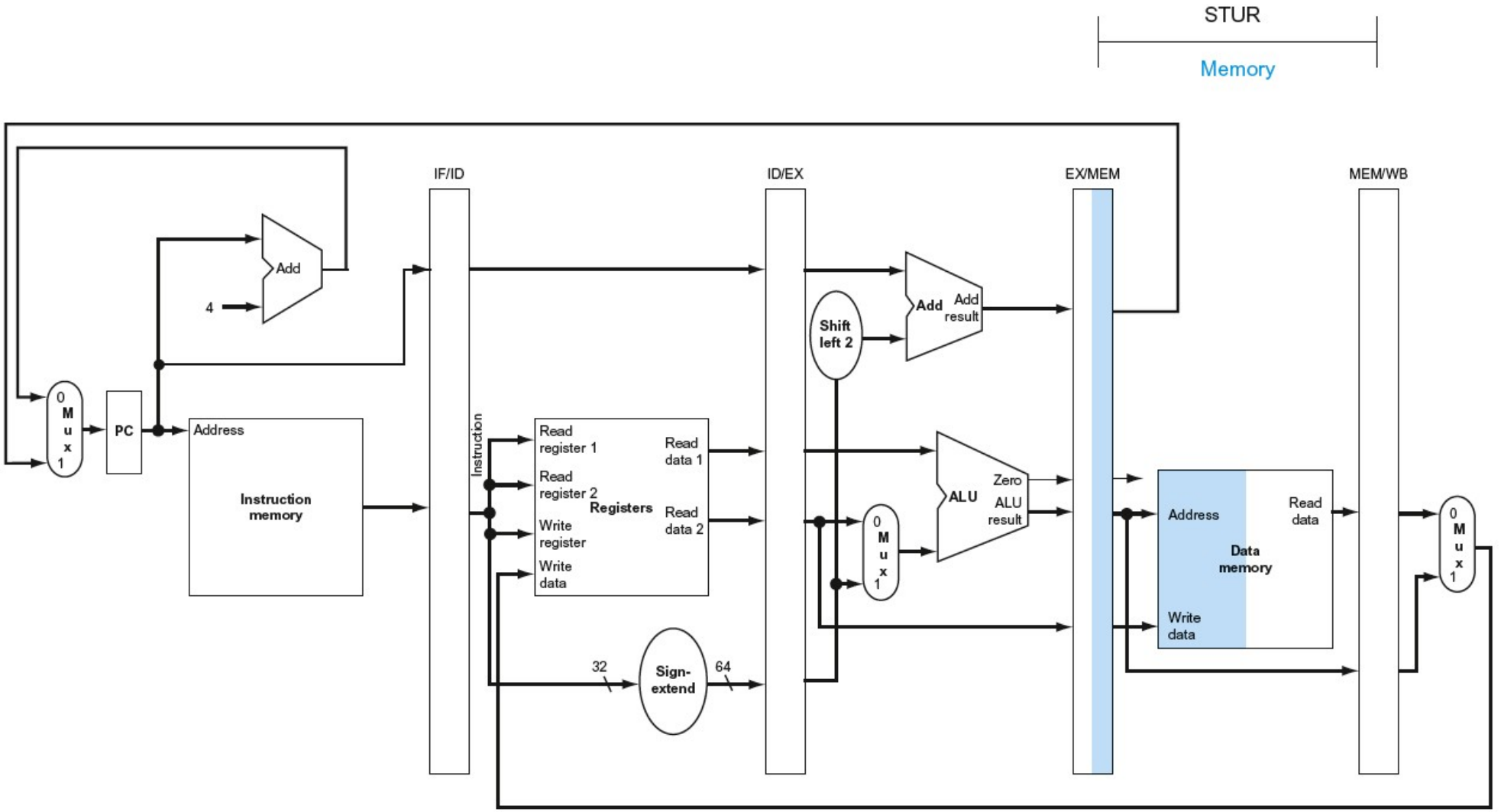
# Corrected Datapath for Load



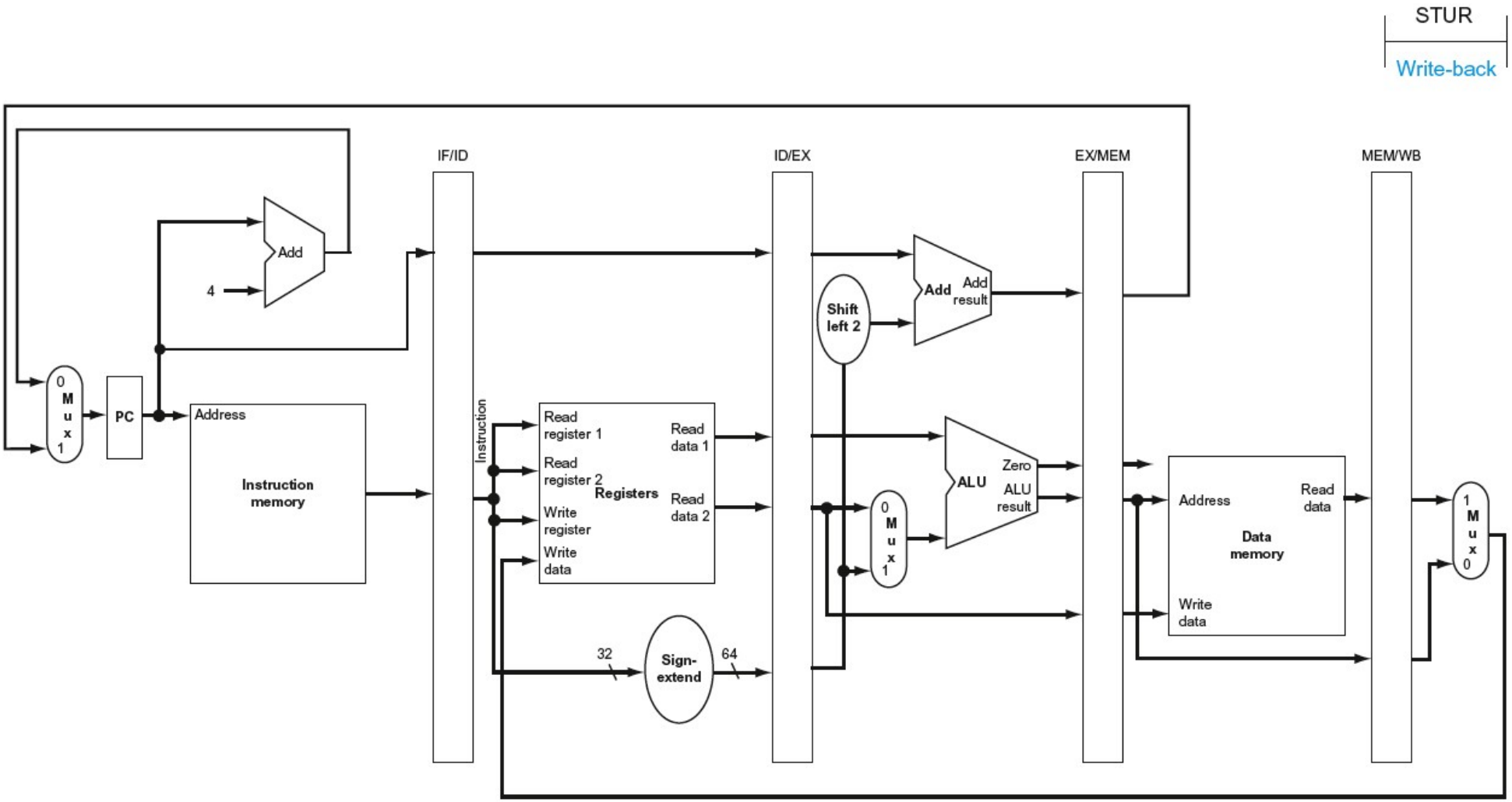
# EX for Store


STUR
Execution
IF/ID
ID/EX
EX/MEM
MEM/WB
Add
4
M u x
0
1
PC
Address
Instruction memory
Instruction
Read register 1
Read register 2
Registers
Write register
Write data
Read data 1
Read data 2
Shift left 2
Add
Add result
ALU
Zero
ALU result
Address
Read data
Data memory
Write data
32
Sign-extend
64

# MEM for Store

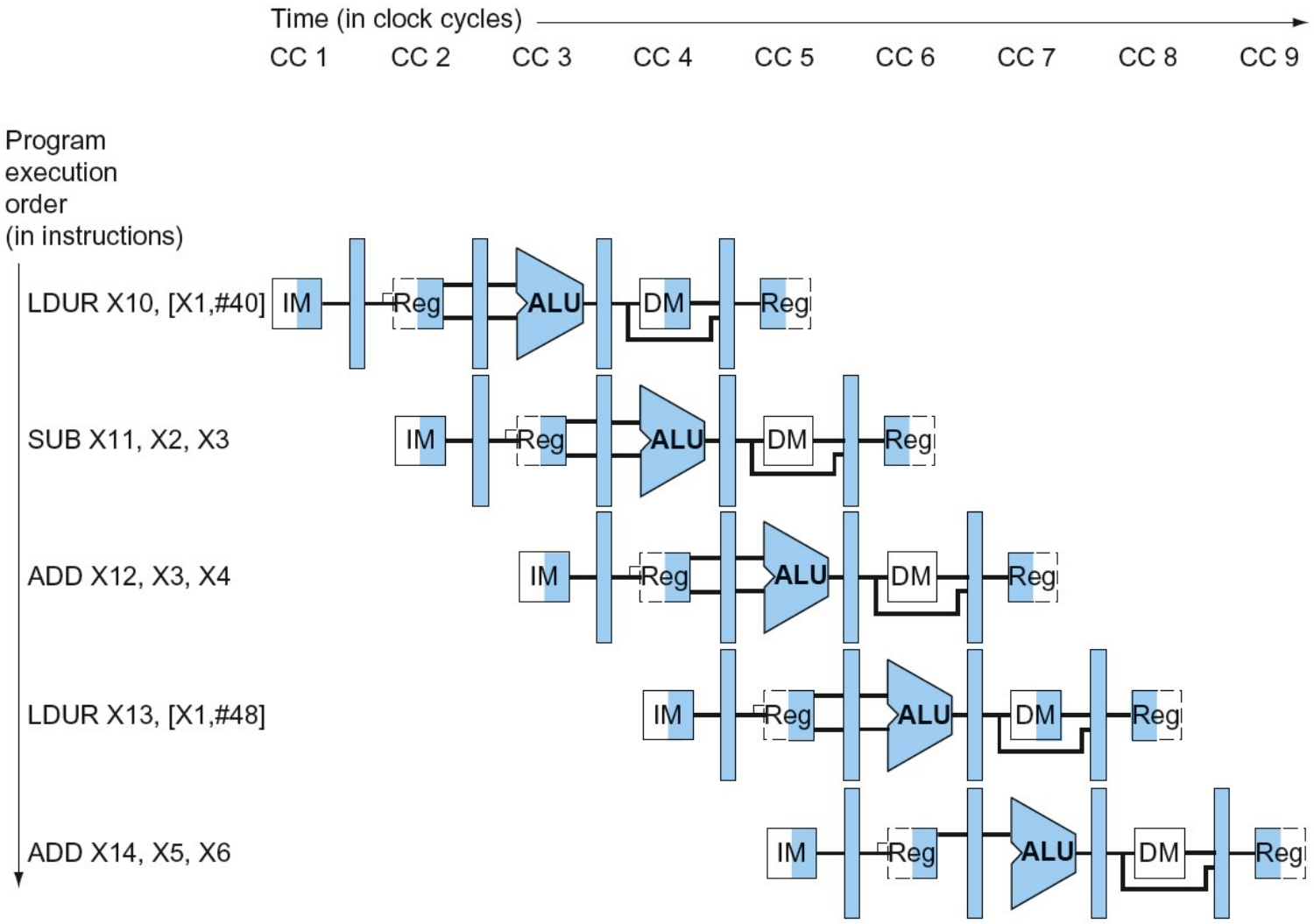


# WB for Store


STUR
Write-back
IF/ID
ID/EX
EX/MEM
MEM/WB
Add
4
M u x
0
1
PC
Address
Instruction memory
Instruction
Read register 1
Read register 2
Registers
Write register
Write data
Read data 1
Read data 2
Shift left 2
Add
Add result
ALU
Zero
ALU result
Address
Read data
Data memory
Write data
32
Sign-extend
64

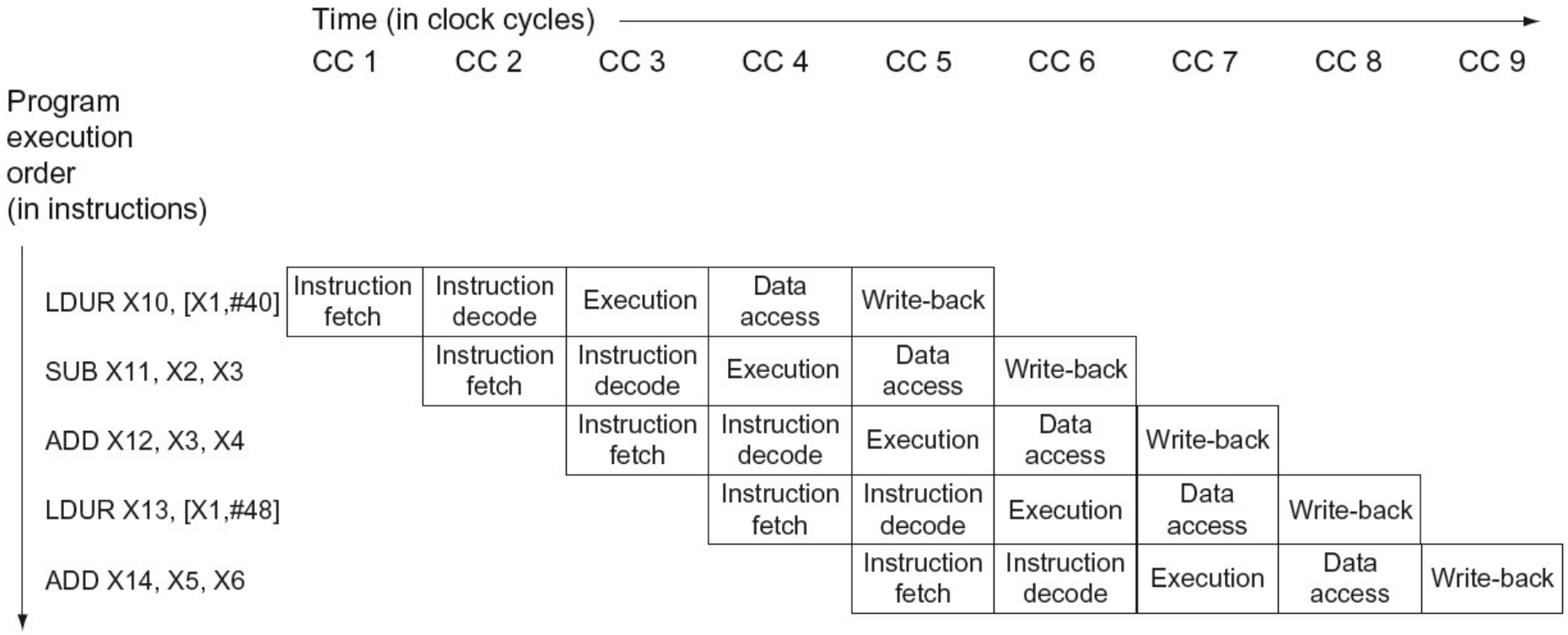
# Multi-Cycle Pipeline Diagram

- Form showing resource usage



# Multi-Cycle Pipeline Diagram

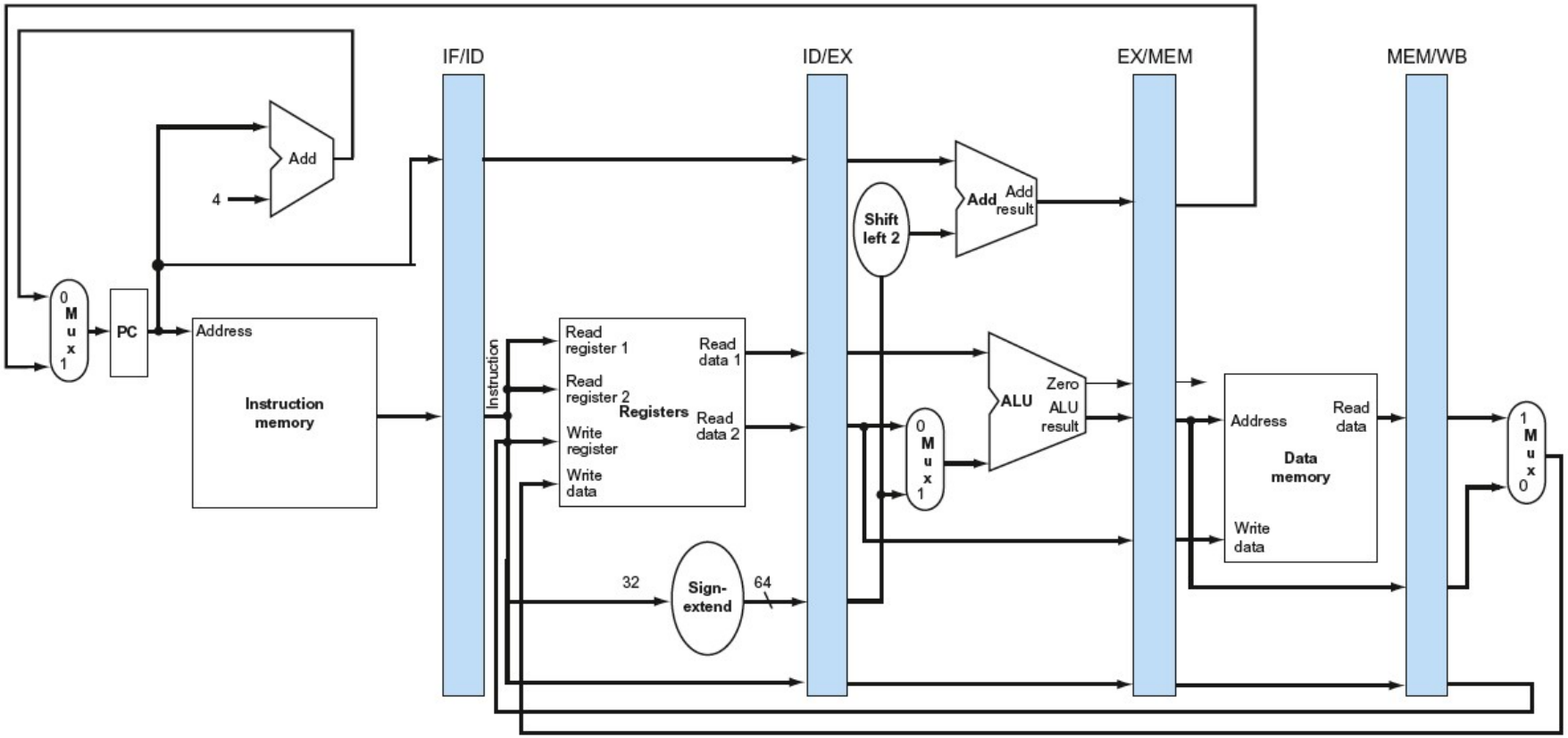
- Traditional form



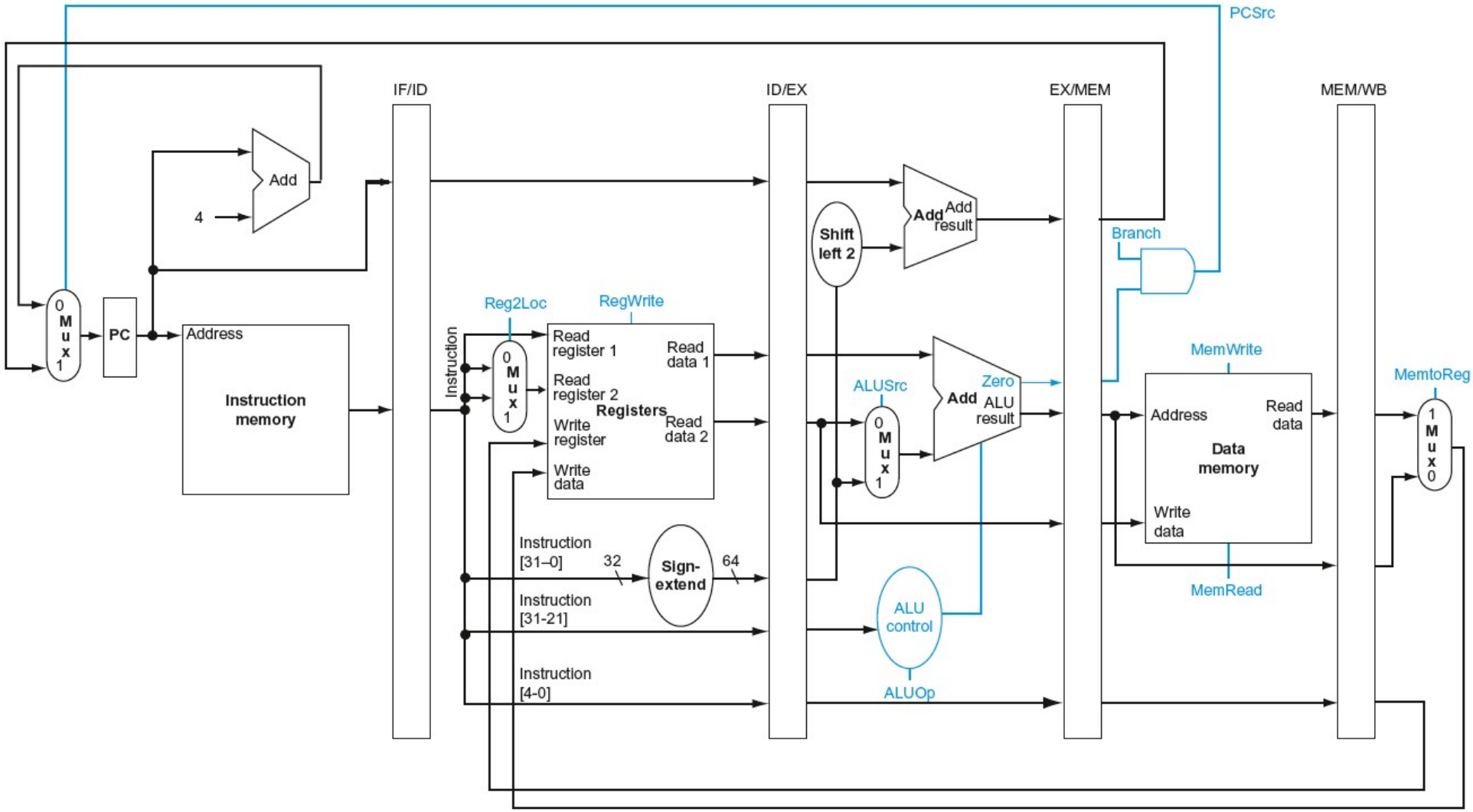
# Single-Cycle Pipeline Diagram

- State of pipeline in a given cycle

| ADD X14, X5, X6 | LDUR X13, [X1,48] | ADD X12, X3, X4 | SUB X11, X2, X3 | LDUR X10, [X1,40] |
|---|---|---|---|---|
| Instruction fetch | Instruction decode | Execution | Memory | Write-back |

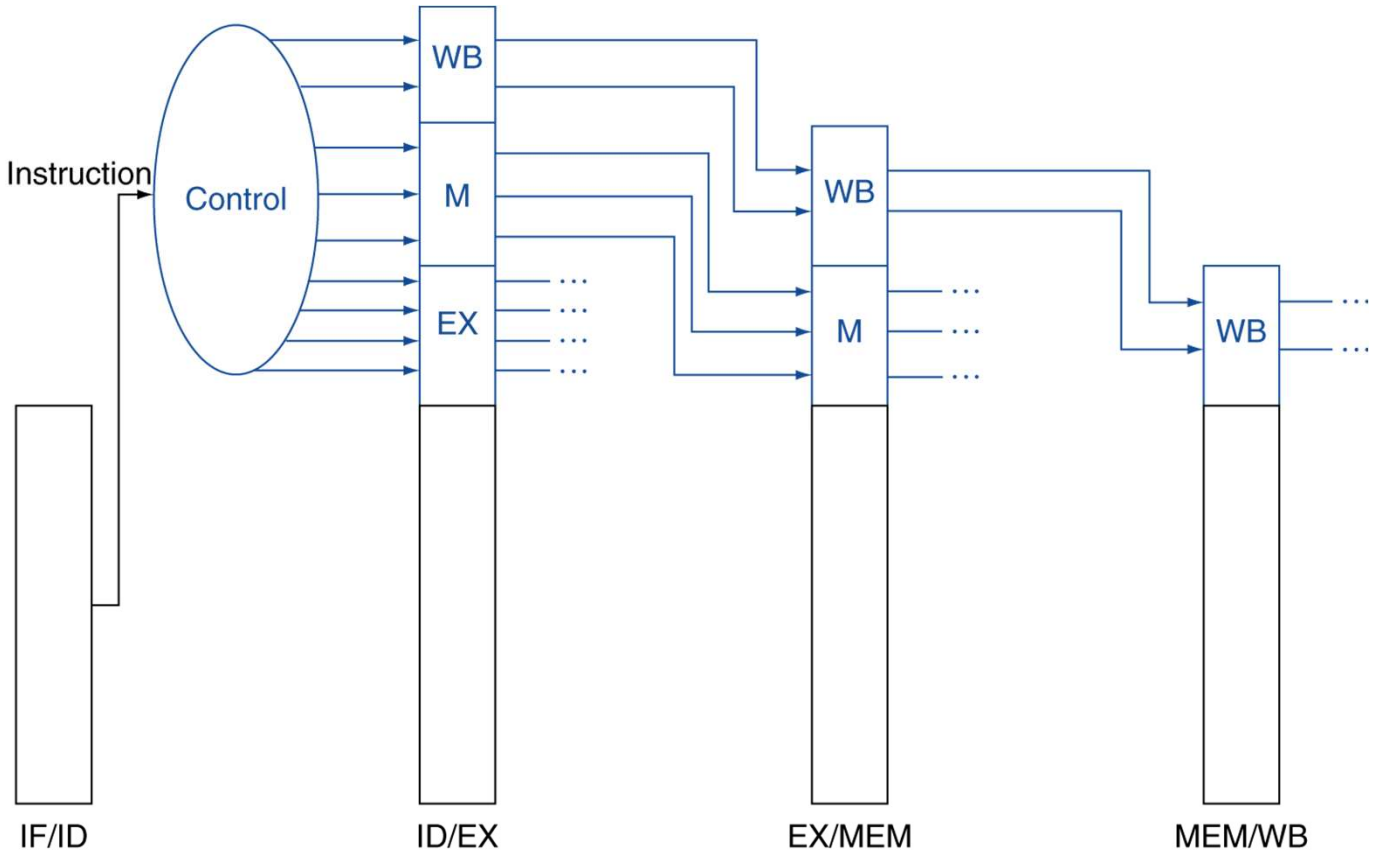


# Pipelined Control (Simplified)

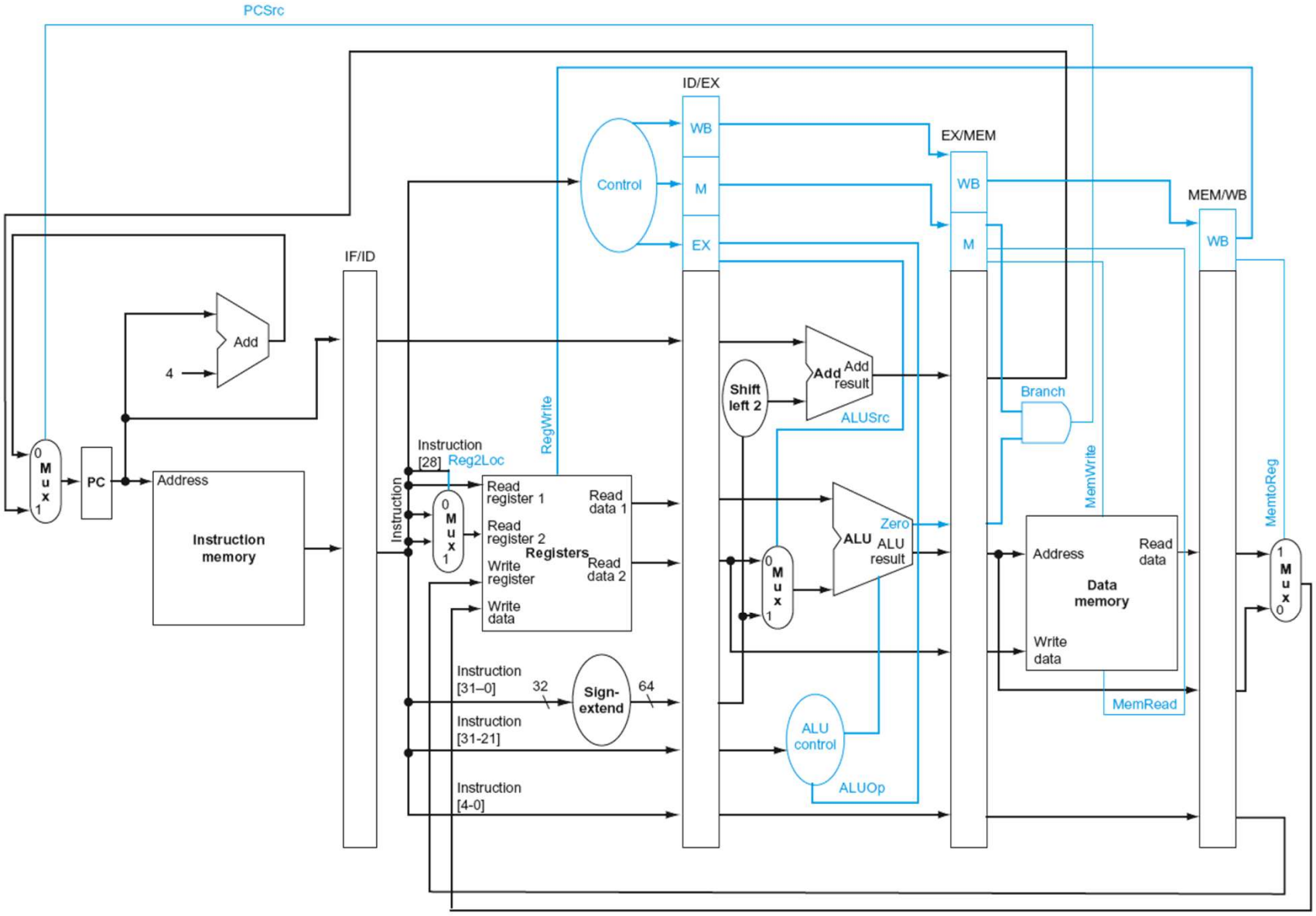


# Pipelined Control

- Control signals derived from instruction
  - As in single-cycle implementation



# Pipelined Control



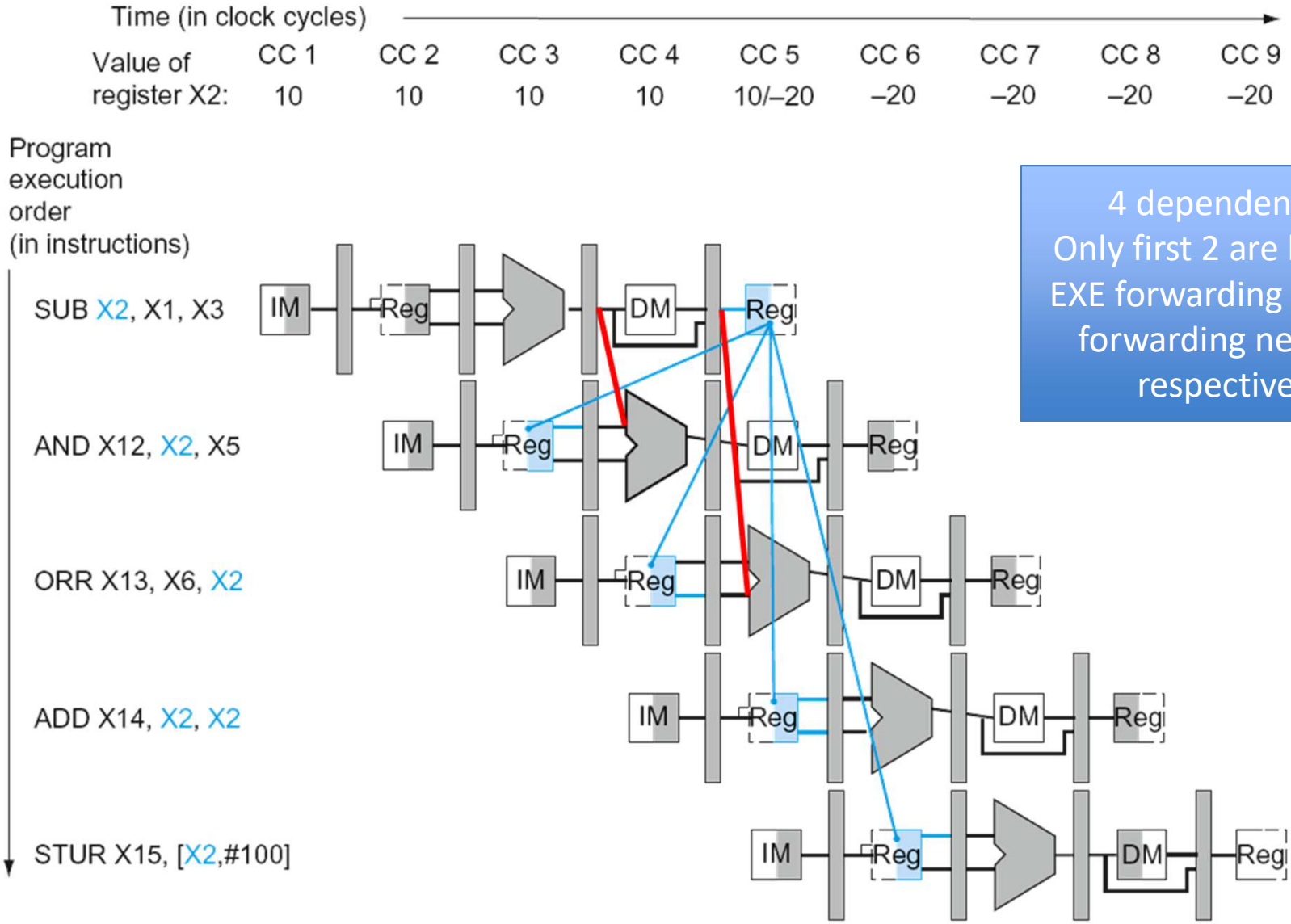
# Data Hazards in ALU Insts

- Consider this sequence:

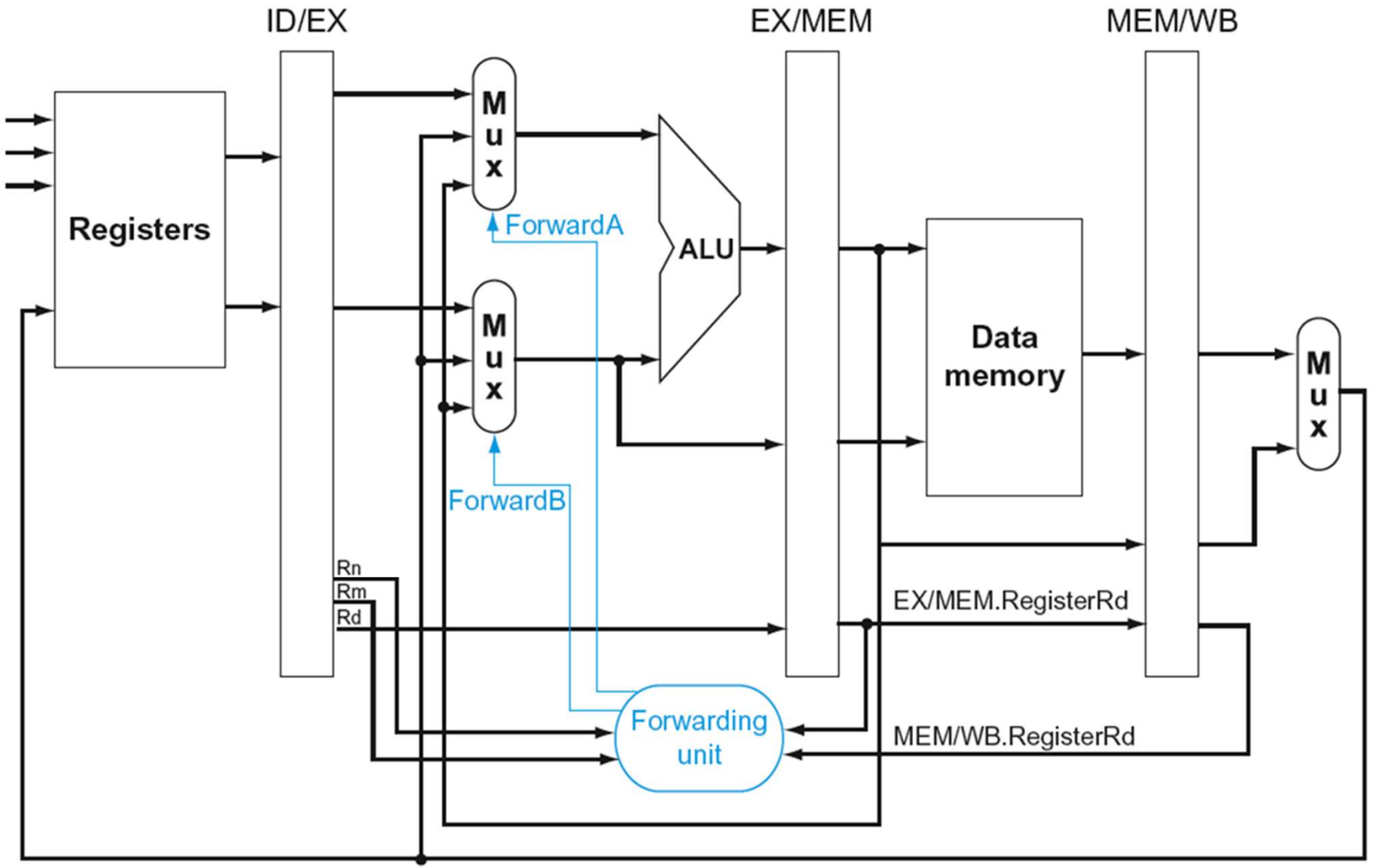
```
SUB  X2, X1,X3
AND  X12,X2,X5
OR   X13,X6,X2
ADD  X14,X2,X2
STUR X15,[X2,#100]
```

- We can resolve hazards with forwarding
  - How do we detect when to forward?

# Dependencies & Forwarding



# Forwarding Paths



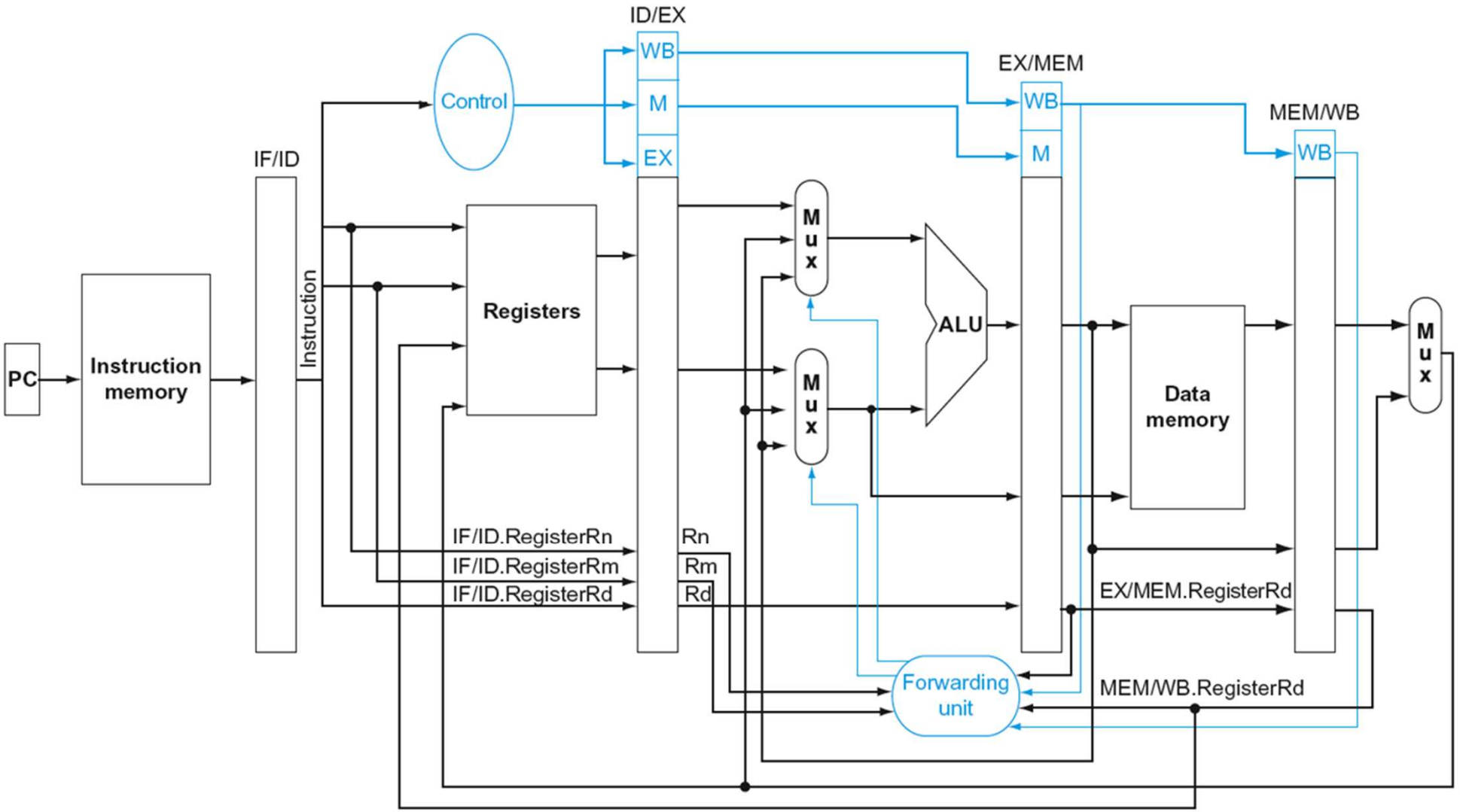
# Double Data Hazard

- Consider the sequence:

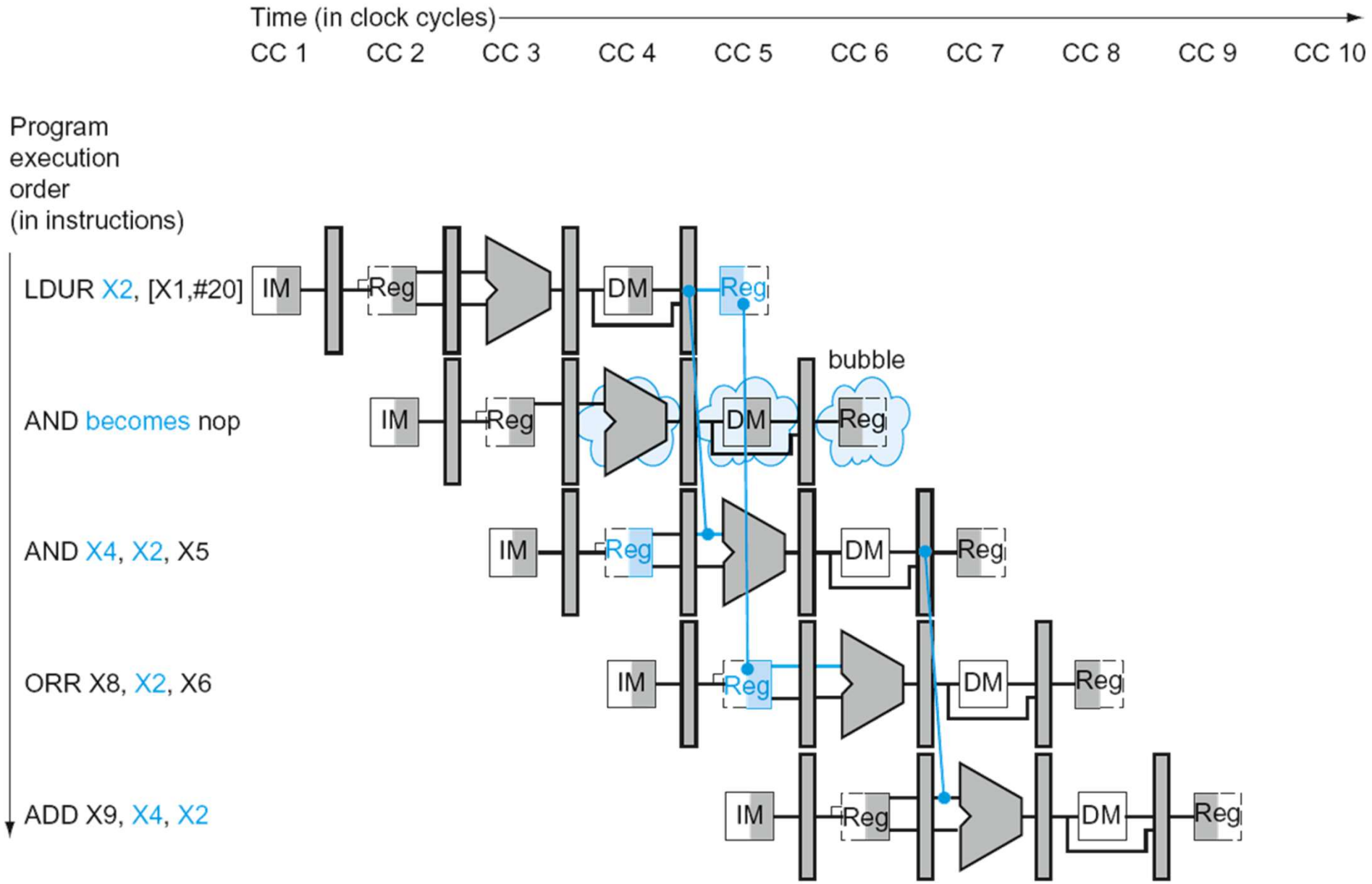
```
add x1,x1,x2
add x1,x1,x3
add x1,x1,x4
```

- Both hazards occur
  - Want to use the most recent

# Datapath with Forwarding



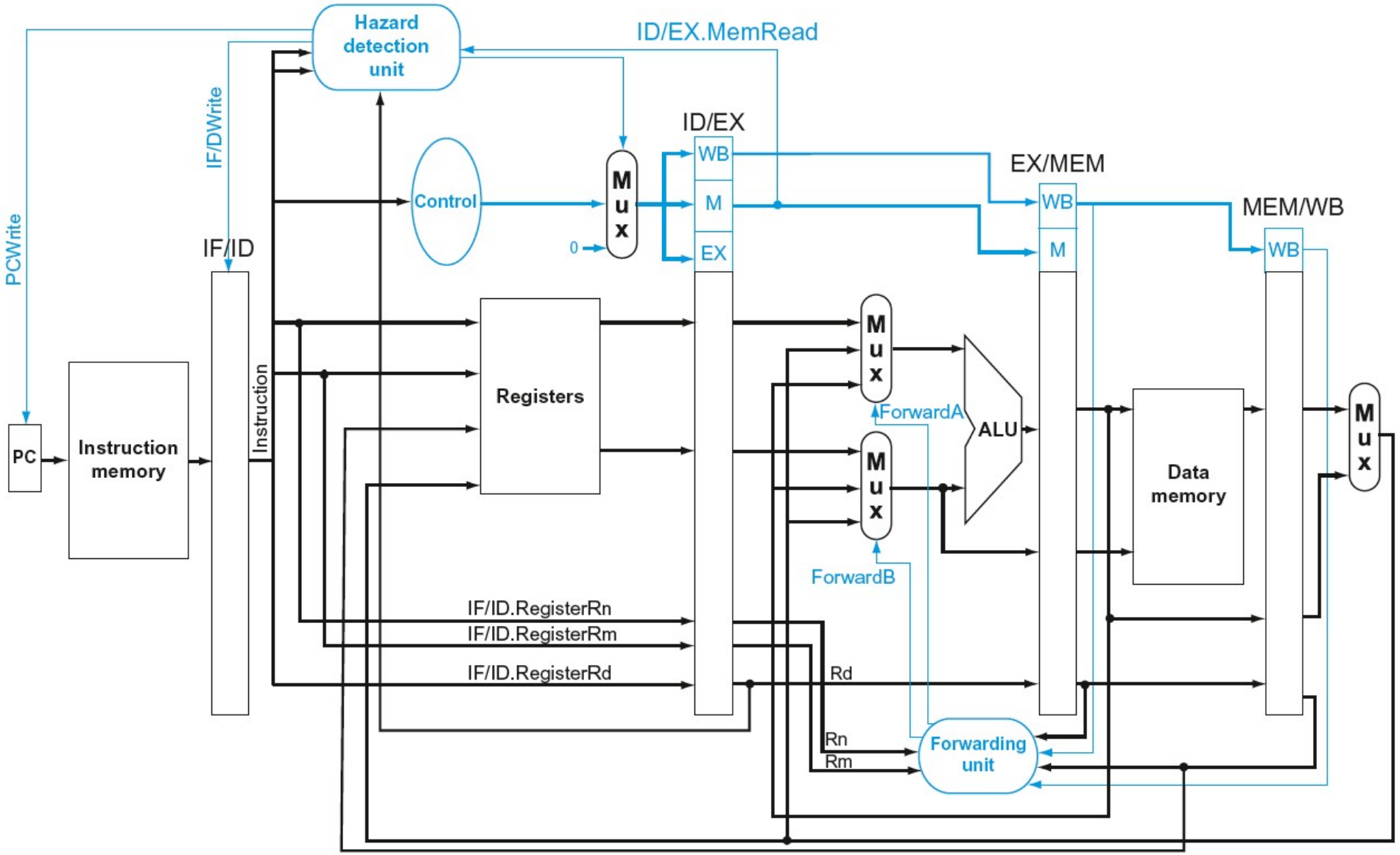
# Load-Use Data Hazard



# How to Stall the Pipeline

- Force control values in ID/EX register to 0
  - EX, MEM and WB do nop (no-operation)
- Prevent update of PC and IF/ID register
  - Using instruction is decoded again
  - Following instruction is fetched again
  - 1-cycle stall allows MEM to read data for LDUR
    - Can subsequently forward to EX stage

# Datapath with Hazard Detection



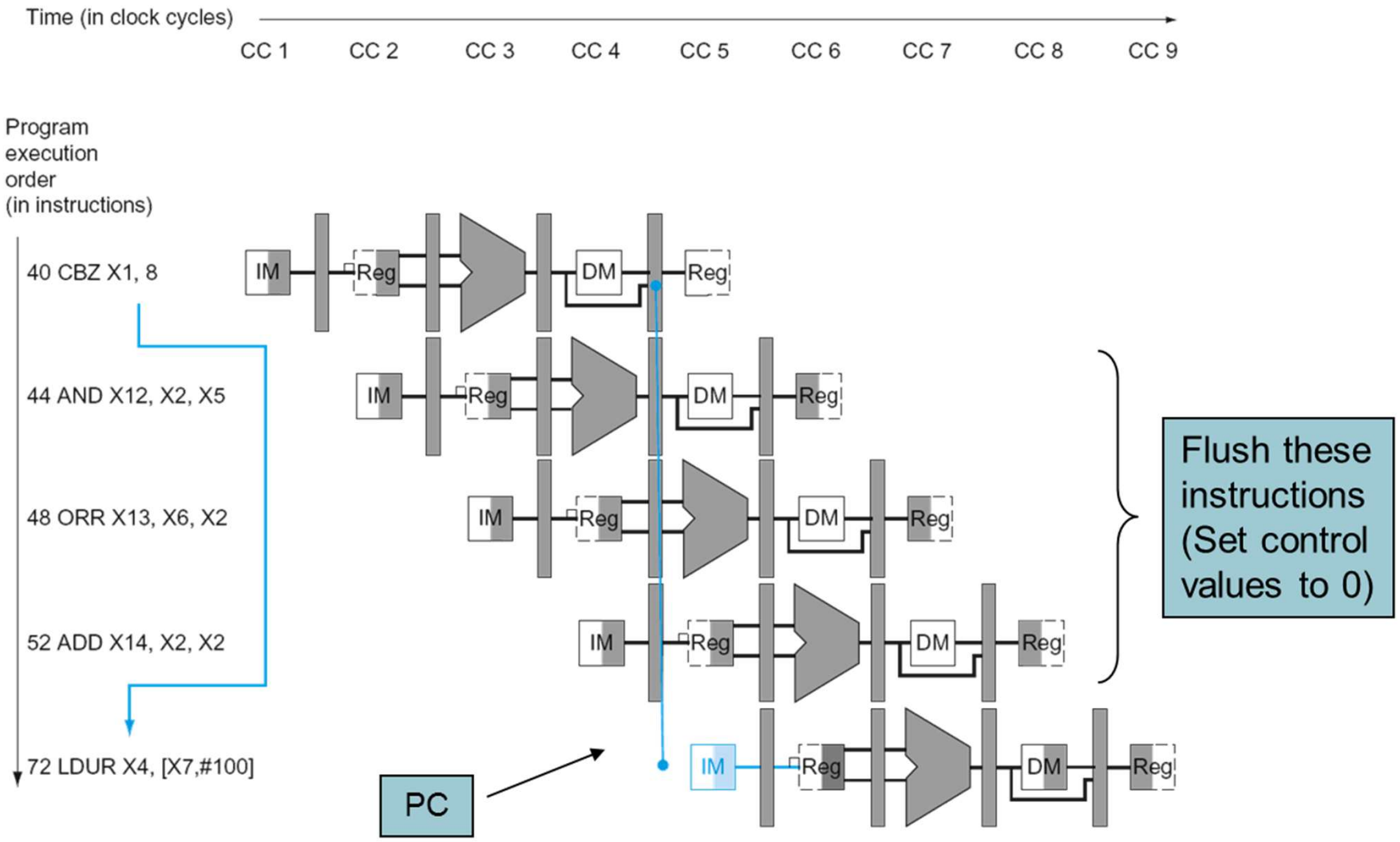
# Stalls and Performance

**The BIG Picture**

- Stalls reduce performance
  - But are required to get correct results
- Compiler can arrange code to avoid hazards and stalls
  - Requires knowledge of the pipeline structure

# Branch Hazards

- If branch outcome determined in MEM



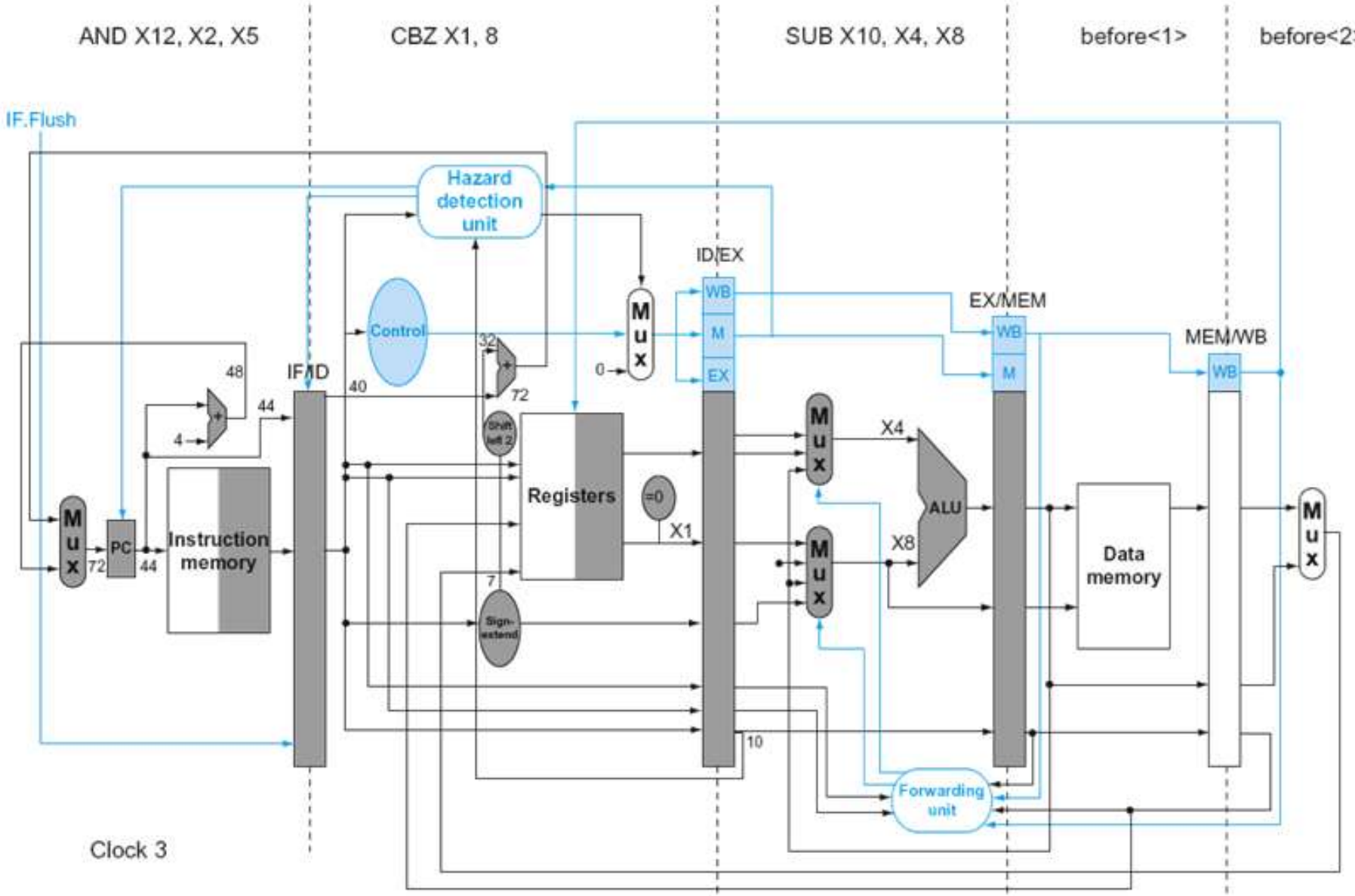
# Reducing Branch Delay

- Move hardware to determine outcome to ID stage
  - Target address adder
  - Register comparator
- Example: branch taken

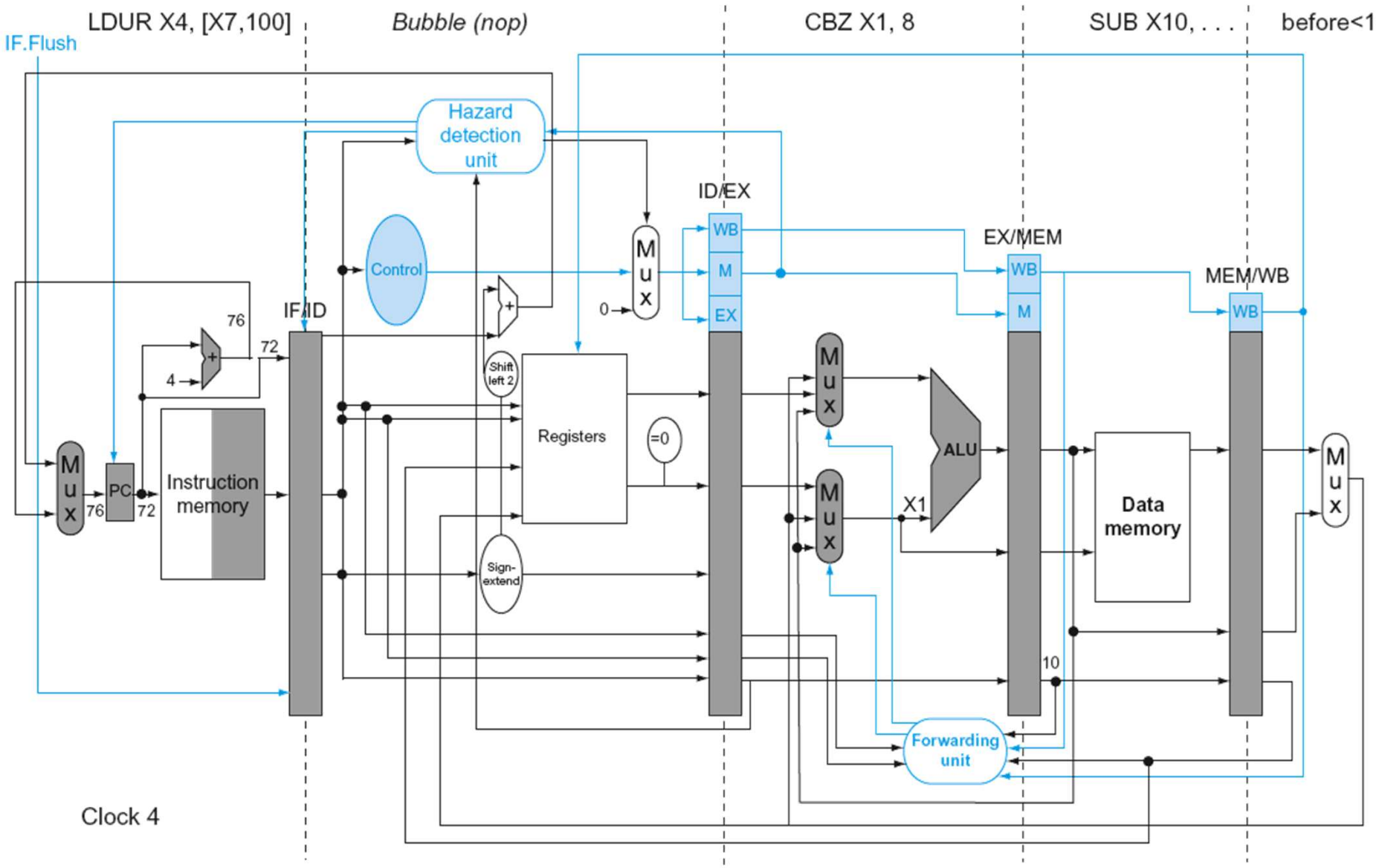
```
36:   SUB  X10, X4, X8
40:   CBZ  X1,  X3, 8
44:   AND  X12, X2, X5
48:   ORR  X13, X2, X6
52:   ADD  X14, X4, X2
56:   SUB  X15, X6, X7
      ...
72:   LDUR X4, [X7,#50]
```

8 instructions to skip =
8 x 4 = 32 bytes to skip

# Example: Branch Taken



# Example: Branch Taken

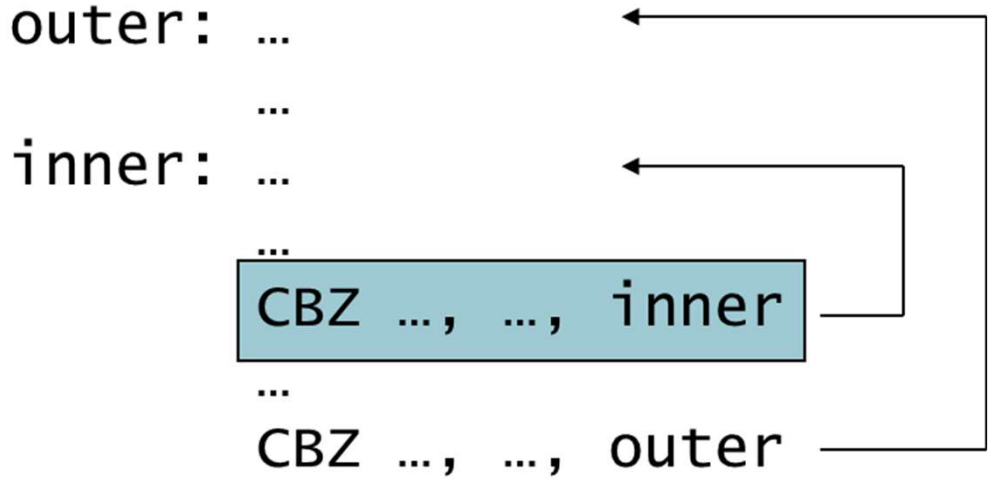


# Dynamic Branch Prediction

- In deeper pipelines, branch penalty is more significant
- Use dynamic prediction
  - Branch prediction buffer (aka branch history table)
  - Indexed by recent branch instruction addresses
  - Stores outcome (taken/not taken)
  - To execute a branch
    - Check table, expect the same outcome
    - Start fetching from fall-through or target
    - If wrong, flush pipeline and flip prediction

# 1-Bit Predictor: Shortcoming

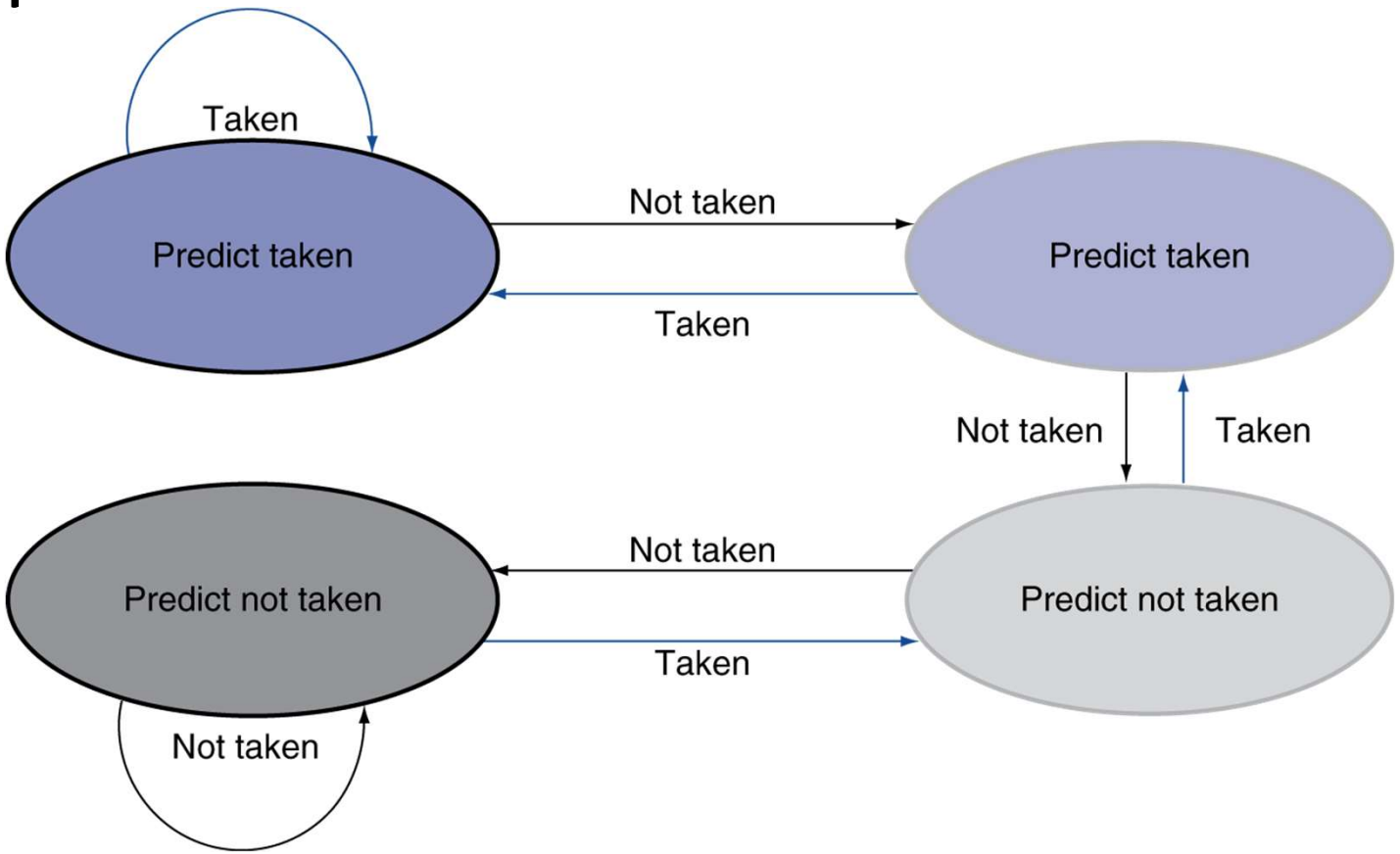
- Predict based on previous outcome
- Inner loop branches mispredicted twice!




  - Mispredict as taken on last iteration of inner loop
  - Then mispredict as not taken on first iteration of inner loop next time around

# 2-Bit Predictor

- Only change prediction on two successive mispredictions



# Instruction-Level Parallelism (ILP)

- Pipelining: executing multiple instructions in parallel
- To increase ILP
  - Deeper pipeline
    - Less work per stage ⇒ shorter clock cycle
  - Multiple issue (aka superscalar)
    - Replicate pipeline stages ⇒ multiple pipelines
    - Start multiple instructions per clock cycle
    - CPI < 1, so use Instructions Per Cycle (IPC)
    - E.g., 4GHz 4-way multiple-issue
      - 16 BIPS, peak CPI = 0.25, peak IPC = 4
    - But dependencies reduce this in practice

# Concluding Remarks



- ISA influences design of datapath and control
- Datapath and control influence design of ISA
- Pipelining improves instruction throughput using parallelism
  - More instructions completed per second
  - Latency for each instruction not reduced
- Hazards: structural, data, control